Mapa conceptual — Sistemas Operativos

UTN FRM — Mapa acumulativo · Unidades 1 y 2 · Las referencias 📖 son del Tanenbaum (páginas impresas); las 🗎 son del PDF de repaso de cada unidad.

Cómo usar este mapa

Mapa conceptual ACUMULATIVO de Sistemas Operativos (UTN FRM). Unidades incluidas: 1 y 2. Última ampliación: 3 de noviembre de 2026.

Este mapa sirve para dos cosas: **estudiar** (tiene el contenido completo, no un resumen) y **ordenar el repaso** (ves de un vistazo qué entra y en qué orden). Cada rama lleva **dos referencias**: la del libro, para leer la fuente, y la del PDF de repaso, para volver a lo que ya leíste.

- **Leyenda de marcas**
  - 📖 **Referencia al Tanenbaum** — capítulo y páginas impresas del libro (la página del PDF del libro es la impresa + 30).
  - 🗎 **Referencia al PDF de repaso** — página real de `repaso_u01.pdf` o `repaso_u02.pdf` y el título de la sección, para volver a lo que ya leíste.
  - 🔢 **Secuencia** — un procedimiento con pasos numerados, para recitarlo en orden.
  - 📯 **Nota de memoria** — mnemotecnia o regla rápida.
  - ⚖ **Par que se confunde** — dos cosas parecidas que hay que separar.
  - 📌 **Trampa de examen** — lo que la cátedra pregunta para que te equivoques.
  - ⚠ **Error que ya cometí** — para no repetirlo.
  - 🔗 **Conexión entre unidades** — se puede hacer clic para saltar a la otra rama.
  - **? Duda que tuve** — preguntas que hice (en el chat o en `googleia.md`) con la respuesta corta.
- **Cómo repasar con el mapa**
  1. Leé la rama de un tema de arriba abajo, en voz alta si podés.
  2. Recitá las secuencias 🔢 sin mirar y después verificá.
  3. Repasá todas las ⚠ y 📌 juntas antes del examen (en la página interactiva está el filtro "Solo errores").
  4. Si una rama no te sale, andá a la página 🗎 del repaso y releé esa sección.
- **Regla de mantenimiento**: el mapa no se reemplaza nunca. En cada unidad nueva se agrega su rama y se reconectan sus conceptos con los de las unidades anteriores. Si se regenera un `repaso_uNN.pdf`, se vuelve a correr `_trabajo/indice_repaso.py` y se actualizan las páginas 🗎.

La idea que une todo

La misma tensión aparece en toda la materia. Si la reconocés, la mitad de las preguntas de desarrollo se contestan solas.

- 📯 **Compartir es rápido pero peligroso; aislar es seguro pero caro.**
  - Monolítico contra microkernel (→ U1: Tema 9 — Estructuras de los sistemas operativos).
  - Hilos contra procesos (→ U2: Tema 3 — Hilos: el modelo clásico).
  - Memoria compartida contra paso de mensajes (→ U2: Tema 7.4b — Paso de mensajes y barreras).
- 📯 **Un candado grande es correcto y lento; los candados chicos e independientes son correctos y rápidos.**
  - El mutex de toda la mesa de los filósofos contra el `estado[]` de Dijkstra (→ U2: 9.1 — Los filósofos comelones).
  - SMP con un mutex global contra SMP con un mutex por subsistema (→ U2: 10.2 — Los tres tipos de SO multiprocesador).
  - Una cola global de listos contra una cola por CPU (→ U2: 10.4 — Planificación de

multiprocesadores).

- 📯 **Cuando no podés avanzar, ¿en qué estado te quedás?** Girar = EN EJECUCIÓN (gasta CPU), ceder = LISTO, dormir = BLOQUEADO (no gasta CPU). Es la pregunta de los temas 6 y 7 de la U2 (→ U2: Los tres niveles de espera).
- 📯 **El algoritmo óptimo siempre necesita un dato del futuro.** SJF necesita las ráfagas (→ U2: 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF), el algoritmo de retrospección necesita saber cuánto dura el candado (→ U2: 10.3 — Sincronización de multiprocesadores), y en la U3 el reemplazo óptimo de páginas necesita las referencias futuras.
- 📯 **Mecanismo contra política.** El mecanismo dice *cómo* se puede hacer algo; la política dice *quién y cuándo*. Aparece en el microkernel (→ U1: Tema 9 — Estructuras de los sistemas operativos) y en la planificación (→ U2: 8.5 — Política contra mecanismo).

U1 — Introducción, hardware y estructura de los SO

📖 Tanenbaum, cap. 1 (págs. 1–72) + 2.1.7 (págs. 93–95) + 5.2 (págs. 343–348) + apuntes de la cátedra (interrupciones del 8086, comandos de Linux)

🗐 Repaso U1, 56 págs. — la clase completa; mapa de la unidad en la pág. 1; resumen condensado de toda la unidad desde la pág. 38

La idea que atraviesa la unidad: el SO existe para **esconder un hardware incómodo** y para **repartirlo entre varios que lo quieren a la vez**. Los temas 2 a 5 son el hardware que hay que esconder, el tema 6 es la puerta por donde se pide, y los temas 7 a 9 cuentan cómo se fue organizando por dentro.

Tema 1 — ¿Qué es un sistema operativo?

📖 cap. 1.1, págs. 3–7; cap. 2.1.7, págs. 93–95

🗐 Repaso U1, pág. 1 — "Tema 1 — ¿Qué es un sistema operativo?" (resumen condensado: pág. 38)

- **Definición difícil** — "el software que se ejecuta en modo kernel", y ni siquiera eso es siempre cierto. El problema es que cumple **dos funciones que no tienen relación entre sí**.
- **Función 1 — Máquina extendida** — el hardware a nivel de lenguaje máquina es **primitivo y complejo de programar**; el SO ofrece **abstracciones** limpias.
  - **Abstracción** — la clave para manejar la complejidad. La que todo el mundo entiende es el **archivo**.
  - Ejemplo: leer un archivo sin SO son 7 pasos de hardware (encender el motor, esperar velocidad, mover el brazo, esperar el sector, leer bits, verificar errores, apagar); con SO es un solo `read(fd, buf, nbytes)`.
  - Cita: *"El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los objetos abstractos entonces creados."*
  - **? "Primitiva" no quiere decir antigua**: quiere decir que tiene el mínimo nivel de abstracción (mover, sumar, comparar, saltar). No sabe qué es un texto ni una pantalla.
- **Función 2 — Administrador de recursos** — **asignación ordenada y controlada** de procesadores, memorias y dispositivos de E/S entre los programas que compiten por ellos.
  - **Multiplexaje en el TIEMPO** — los recursos se turnan (CPU, impresora). Genera el problema de *¿quién va primero?* → 🔑 planificación (→ U2: Tema 8 — Planificación de la CPU ★).
  - **Multiplexaje en el ESPACIO** — cada uno recibe una parte (memoria principal, disco). Genera el problema de *¿y si uno pisa al otro?* → protección (U3).
  - ⚠ **El disco tiene dos recursos adentro**: la superficie se reparte en el **espacio**, pero el **brazo**, que es uno solo, se reparte en el **tiempo** (de ahí sale la planificación de disco de la U5). En el cuestionario me faltó identificar qué se multiplexa de cada forma (parte B-a, 1,5/2).
- **Modelo de la multiprogramación** — si un proceso espera E/S una fracción **p** del tiempo y hay **n** procesos independientes en memoria, la probabilidad de que todos esperen a la vez es p^n.

- 📯 **Uso de la CPU = 1 – pⁿ.** A n se le llama **grado de multiprogramación**.
- Ejemplo con p = 0,8: n=1 → 20 %; n=2 → 36 %; n=3 → 48,8 %; n=4 → 59,04 %; n=5 → 67,23 %; n=10 → 89,26 %.
- El rendimiento marginal **decrece**: cada proceso extra aporta menos y además consume memoria.
- 📌 El modelo ingenuo ("si calcula el 20 %, con 5 procesos la CPU está al 100 %") es **demasiado optimista** porque supone que los 5 nunca esperan E/S al mismo tiempo.
- 🔗 Es la justificación numérica de por qué existe la multiprogramación (→ U1: Tema 7 — Historia por generaciones) y por qué el planificador elige entre los LISTOS (→ U2: Tema 2 — Estados del proceso y el BCP).

Tema 2 — Repaso del hardware

📖 cap. 1.3, págs. 19–33

🗐 Repaso U1, pág. 3 — "Tema 2 — Repaso del hardware" (resumen condensado: pág. 39)

- **La CPU y sus registros**
  - 🔢 **Ciclo básico de toda CPU**
    1. **Obtener** la instrucción de memoria (en la dirección que indica el PC).
    2. **Decodificarla** para saber su tipo y sus operandos.
    3. **Ejecutarla**.
    4. Volver al paso 1.
  - **Contador de programa (PC)** — dirección de la **siguiente** instrucción a obtener. Al SO le importa porque es lo que hay que guardar para suspender y reanudar un proceso.
  - **Apuntador de pila (SP)** — tope de la pila: marcos de los procedimientos llamados y todavía no retornados. Cada proceso (y cada hilo) tiene su propia pila.
  - **PSW (palabra de estado del programa)** — bits de código de condición (resultado de la última comparación) **y el bit de modo** (kernel/usuario). Ahí vive la protección.
  - **? CPU, procesador y núcleo**: la CPU es el procesador (el chip que ejecuta instrucciones). Un **núcleo** es una CPU completa dentro del chip; un chip multinúcleo tiene varias.
  - **? Ciclo de reloj** — el "tic" que marca el ritmo de la CPU; a 3 GHz dura unos 0,33 ns. "La L2 cuesta 1 o 2 ciclos" quiere decir 1 o 2 de esos tics (complemento elaborado).
  - **? De dónde saca la instrucción**: de la **memoria principal (RAM)**, en la dirección que indica el PC. Si la línea está en caché, sale de la caché, que es más rápida.
  - ⚖ **PSW contra campo "estado" del BCP**: el PSW es un **registro del hardware** (bits de condición y modo). El estado del BCP es un **dato del SO** (en ejecución, listo o bloqueado). Son cosas distintas que comparten la palabra "estado" (→ U2: BCP (bloque de control de proceso)).
- **Modo kernel y modo usuario** — el cimiento de toda la materia.
  - **Modo usuario** — solo un **subconjunto** de instrucciones y de hardware. No puede deshabilitar interrupciones, tocar los registros de E/S ni acceder a memoria ajena.
  - **Modo kernel** — **cualquier** instrucción y **todo** el hardware. Ahí corre el SO.
  - **TRAP** — la única puerta legal para pasar de usuario a kernel (→ U1: Tema 6 — Llamadas al sistema).
  - 📌 **Instrucción privilegiada ejecutada en modo usuario**: el **hardware** mira el bit de modo del PSW y la **rechaza antes de ejecutarla**, generando un trap. Recién ahí el SO toma el control y, normalmente, mata al proceso. **La protección la impone el hardware; el SO la configura.**
  - ⚠ Error de la clase: decir que "el SO se cuelga y después reacciona". No: el hardware la frena antes.
  - 🔗 Sin estos dos modos no habría multiprogramación segura (→ U1: Tema 7 — Historia por generaciones) ni estructuras de SO (→ U1: Tema 9 — Estructuras de los sistemas operativos).
- **Jerarquía de memoria**

    - De arriba abajo: **registros** (igual que la CPU, < 1 KB) → **caché L1/L2** (1 a 2 ciclos, KB a MB) → **RAM** (decenas de ns, GB) → **disco** (milisegundos, TB, no volátil) → **cinta** (segundos). Hacia arriba: más rápido, más caro y menos capacidad.
    - **Volátil** — se borra sin energía (registros, caché, RAM). El disco y la cinta no son volátiles.
    - **Línea de caché** — la memoria se divide en líneas; las más usadas se guardan en la caché, dentro o muy cerca de la CPU.
    - **Acierto de caché** — la línea está en la caché y no se va al bus. **Fallo de caché** — no está, y se paga el acceso completo.
    - **Localidad de referencia** — los programas tienden a usar direcciones cercanas a las que acaban de usar. Por eso la caché funciona.
    - **?** Mi reformulación ("la localidad es lo que se guarda en caché para no ir a la RAM") mezcla dos cosas: la **localidad** es una *propiedad del programa*; la **caché** es el *hardware* que la aprovecha.
    - 🔢 **Tiempo medio de acceso con caché** (L1 = 1 ns, RAM = 10 ns, 95 % de aciertos)
        1. 95 % de las veces está en la caché → 1 ns.
        2. 5 % de las veces es un fallo → 1 ns (mirar) + 10 ns (RAM) = 11 ns.
        3. Promedio: Tm = 0,95 × 1 + 0,05 × 11 = 1,50 ns.
        4. Sin caché serían 10 ns → la caché es 6,67 veces más rápida.
        5. Con mala localidad (50 % de aciertos): Tm = 6,00 ns → solo 1,67 veces más rápida. **La caché vale lo que vale la localidad del programa.**
    - 🔗 La TLB de la U3 es una caché de traducciones: misma idea, mismo cálculo de tiempo medio. Y la caché vuelve en los multiprocesadores, con la coherencia (→ U2: 10.1 — Hardware de multiprocesador).
- **Memorias no volátiles** — **ROM** (de fábrica, no se modifica); **EEPROM y flash** (se reescriben); **CMOS** (volátil, con batería: fecha, hora y configuración del BIOS, incluida la lista de dispositivos de arranque).
    - **? El BIOS no está en la RAM ni en el disco**: está en un chip ROM/flash de la placa madre. Por eso puede arrancar sin disco (→ U1: Tema 4 — El arranque y el sector de booteo).
    - **? Disco contra RAM**: la RAM es rápida y volátil (trabajo); el disco es lento y permanente (almacenamiento).
- **Disco** — platos que giran a 5400, 7200 o 10800 rpm. **Pista** = anillo que lee una cabeza; **cilindro** = todas las pistas bajo una misma posición del brazo; **sector** = la unidad mínima de lectura.
- **MMU (unidad de administración de memoria)** — la parte de la CPU que traduce la dirección del programa a la dirección física. Es el hardware de la memoria virtual (U3).
- **Dispositivos de E/S** — siempre dos partes: el **dispositivo** y su **controlador** (un chip). El software que le habla al controlador es el **driver**, y corre en modo kernel.
    - ⚖ **Driver contra controlador**: el **controlador** es *hardware* (el chip del dispositivo); el **driver** es *software* (el programa del SO que le habla a ese chip). En castellano a veces al driver le dicen "controlador": por eso se confunden.
- **Buses** — ISA (viejo) → PCI; USB para dispositivos lentos; SCSI para dispositivos veloces.
- **Plug and play** — el sistema recolecta solo la información de los dispositivos y asigna de forma centralizada los niveles de interrupción y las direcciones de E/S.

Tema 3 — Las tres formas de hacer E/S

📖 cap. 1.3.5, págs. 27–30; cap. 5.2, págs. 343–348

🗎 Repaso U1, pág. 6 — "Tema 3 — Las tres formas de hacer E/S" (resumen condensado: pág. 41)

- 🌕 **Lo más importante**: los tres métodos transfieren los mismos datos y **tardan lo mismo**, porque la duración la impone el dispositivo. Lo que cambia es **cuánta CPU se gasta**.
- 📌 **"¿Cuál es más rápido?"** — ninguno. Es la trampa del tema.
- 🌕 **La pava** (4 minutos para que hierva): 1 te quedás mirándola sin hacer nada (**programada**); 2 la pava

silba cuando hierve y mientras tanto hacés otra cosa, pero si silbara cada 5 segundos no servirías para nada (**interrupciones**); 3 la pava eléctrica se apaga sola y avisa una sola vez al final (**DMA**).

- **1. E/S programada (espera ocupada, polling)** — el driver inicia la operación y entra en un ciclo cerrado **sondeando** continuamente al dispositivo para ver si terminó. La CPU queda ocupada el 100 % del tiempo.
  - **Sondear** — preguntar una y otra vez "¿terminaste?".
  - **?** Para el SO, el **disco rígido, un pendrive, el teclado o la impresora son todos dispositivos de E/S**: cada uno con su controlador y su driver.
  - 🔗 **Es la misma espera ocupada de la U2**: gastar CPU girando hasta que pase algo (→ U2: Tema 6 — Exclusión mutua con espera ocupada).
- **2. E/S por interrupciones** — el driver inicia el dispositivo, le pide que **genere una interrupción** cuando termine, y retorna. El SO bloquea al proceso llamador si hace falta y busca otro trabajo. Se paga **una interrupción por unidad de datos**.
  - 🔢 **Tratamiento de una interrupción por hardware** (fig. 1-11)
    1. El driver le dice al **controlador** del dispositivo qué hacer, escribiendo en sus registros de dispositivo. El controlador arranca el dispositivo.
    2. Al terminar, el controlador avisa al **chip controlador de interrupciones** usando ciertas líneas del bus.
    3. Si el controlador de interrupciones puede aceptarla (puede no hacerlo si está ocupado con una de mayor prioridad), activa un **pin del chip de la CPU**.
    4. El controlador de interrupciones pone el **número del dispositivo** en el bus, para que la CPU sepa cuál terminó.
    5. La CPU **apila el PC y el PSW** y pasa a **modo kernel**.
    6. Usa el número del dispositivo como índice en el **vector de interrupciones** para encontrar la dirección del **manejador**.
    7. El manejador (parte del driver) corre, consulta el estado del dispositivo y, al terminar, se **retorna a la primera instrucción que no se había ejecutado**.
  - **Vector de interrupciones** — una tabla en memoria con la dirección del manejador de cada dispositivo.
  - ⚖ **Vector de interrupciones contra PC**: el PC es un **registro** con la dirección de la próxima instrucción; el vector es una **tabla en memoria** de donde se *carga* el nuevo PC cuando llega una interrupción.
  - **Deshabilitar interrupciones** — la CPU puede postergarlas; el dispositivo sigue señalizando, pero la CPU no se interrumpe hasta que se rehabilitan. 🔗 En la U2 es el primer intento de exclusión mutua (→ U2: Tema 6 — Exclusión mutua con espera ocupada).
  - 🔗 La versión de la U2 de esta secuencia, con el BCP y el planificador, son los 8 pasos de la fig. 2-5 (→ U2: Manejo de una interrupción y cambio de contexto).
- **3. E/S con DMA (acceso directo a memoria)** — un chip especial mueve los datos entre la memoria y el controlador **sin intervención constante de la CPU**. La CPU lo configura (cuántos bytes, qué direcciones, qué operación) y al terminar recibe **una sola interrupción**.
  - ⚖ **DMA no reemplaza a las interrupciones**: reduce su cantidad a **una por transferencia completa**.
- **Canales de datos** — en las mainframes, procesadores de E/S especializados con su propio repertorio de instrucciones, que ejecutan programas de canal completos.
- **Comparación** — CPU libre: programada **no**, interrupciones **sí**, DMA **sí**. Interrupciones: 0 / una por byte o bloque / 1. Hardware extra: ninguno / controlador de interrupciones / chip DMA. Uso típico: dispositivos simples y arranque / teclado y mouse / disco, red y video.
- 🔢 **Ejemplo que se toma** (1000 bytes, un byte cada 100 µs, atender una interrupción cuesta 5 µs)
  1. Duración total = 1000 × 100 µs = 100 ms, **igual en los tres**.
  2. Programada → la CPU está ocupada 100 ms → **100 %**.
  3. Interrupciones → 1000 × 5 µs = 5 ms → **5 %**.

4. DMA → 1 × 5 µs = 0,005 ms → **0,005 %**.

- 🌤 **Teclado con interrupciones, disco con DMA**: 10 teclas/s × 5 µs = 50 µs de CPU por segundo (nada); un disco a 100 MB/s con una interrupción por byte pediría 500 s de CPU por cada segundo (imposible).

Tema 4 — El arranque y el sector de booteo

📖 cap. 1.3.7, pág. 33 (el reparto interno de los 512 bytes es complemento elaborado)

🗐 Repaso U1, pág. 8 — "Tema 4 — El arranque y el sector de booteo" (resumen condensado: pág. 42)

- **El problema del huevo y la gallina** — el SO está en el disco, pero para leer el disco hace falta el SO. La solución es una **cadena de cargadores**, cada uno más grande y capaz que el anterior, y cada uno existe solo para cargar al siguiente.
- **Partición** — una porción del disco con su propia organización interna. Las "paredes" no existen físicamente: son datos anotados en el primer sector.
- 🔢 **Los siete pasos del arranque**
  - Se enciende la máquina. La CPU ejecuta el **BIOS**, que está en ROM/flash en la placa madre (no depende del disco).
  - El BIOS se autoexamina: cuenta la RAM, verifica el teclado, explora los buses ISA y PCI y registra los dispositivos heredados y plug and play.
  - Determina el **dispositivo de arranque** consultando la lista guardada en la **CMOS**.
  - Lee el **primer sector** de ese dispositivo (el **sector de booteo**, MBR), lo carga en memoria y lo ejecuta.
  - El sector de booteo examina su **tabla de particiones**, busca la partición **activa** y lee de ahí el **cargador secundario**.
  - El cargador secundario lee el **sistema operativo** y lo inicia.
  - El SO consulta al BIOS la configuración, carga los drivers en el kernel, inicializa sus tablas, crea los procesos de segundo plano y arranca el login o la GUI.
  - 🌤 **BIOS → MBR → partición activa → cargador secundario → SO.**
  - 📌 **¿Por qué hacen falta dos cargadores?** El código del MBR tiene 446 bytes; un cargador secundario como **GRUB** ocupa **cientos de kilobytes** (unas mil veces más). En 446 bytes solo entra "buscar la partición activa y leer su primer sector".
  - 📌 **El MBR no lee ext4**: solo salta al primer sector de la partición activa. El que entiende el sistema de archivos es el **cargador secundario**.
  - **Si se corrompe el código del MBR**: los archivos quedan intactos pero la máquina no arranca. Se repara **reinstalando el código de arranque del MBR** (`bootrec /fixmbr` en Windows; reinstalar GRUB en Linux).
- **MBR (sector de booteo maestro)** — un sector de **512 bytes**: **446** de código de arranque + **64** de tabla de particiones + **2** de firma **55 AA**.
  - 🔢 **La cuenta**: firma = 2 bytes; tabla = 4 particiones primarias × 16 bytes = 64; código = 512 − 64 − 2 = 446; verificación 446 + 64 + 2 = 512.
  - **Renglón de 16 bytes** — 1 byte "¿es la activa?", 1 byte tipo (NTFS, ext4, FAT32), 4 bytes sector de inicio, 4 bytes cantidad de sectores, 6 bytes otros datos.
  - 🌤 **Los 446 son un programa (instrucciones); los 64 son datos (cuatro renglones).** No se comparan.
  - ⚠ **La tabla no contiene las particiones: las describe.** Una partición de 50 MB o de 2 TB ocupa igual 16 bytes. (Error que cometí: pensar que los 64 bytes "eran" las particiones.)
  - **Firma 55 AA** — sin ella el BIOS considera que el disco no es arrancable; 2 bytes corruptos dejan la máquina sin bootear aunque los archivos estén intactos.
  - **Límite de 2 TB del MBR** — el campo de tamaño tiene 4 bytes: $(2^{32} - 1) \times 512$ bytes ≈ 2 TB. Para superarlo se inventó **GPT**, con campos de 8 bytes.

- ⚖ **CMOS contra sector 0**: la **CMOS** guarda la *lista de dispositivos de arranque* (y la fecha y la hora). El **sector 0** guarda el *código de arranque de 446 bytes* y la tabla de particiones.
    - ⚠ Los confundí en el cuestionario (parte B-d, 1,5/2).
- 🌛 **¿Cómo lee el BIOS el sector 0 si no sabe leer archivos?** "Dame el sector 0" es una orden al hardware, que la controladora entiende sola. "Dame el archivo /boot/vmlinuz" exige conocer el sistema de archivos. El BIOS sabe abrir el libro en la página 237, pero no buscar un capítulo por el índice.
- **? ¿El BIOS ocupa RAM?** No: el BIOS vive en la ROM/flash de la placa. Lo que se carga en RAM son el MBR, el cargador secundario y el SO.

Tema 5 — Ensamblador e interrupciones del 8086

📖 No está en el Tanenbaum: es material de la cátedra — Apunte Interrupciones (1).pdf y Tp5 - Tabla Interrupciones.pdf

🗐 Repaso U1, pág. 13 — "Tema 5 — Ensamblador e interrupciones del 8086" (resumen condensado: pág. 44)

- **El problema** — en ensamblador no hay una instrucción para "mostrar texto": solo mover, sumar, restar, comparar y saltar. Las rutinas ya las escribieron el BIOS y el DOS, y están en memoria desde el arranque.
- 🌛 **Regla de oro: INT elige la FAMILIA, AH elige la FUNCIÓN y los demás registros llevan los datos.**
    - 📌 Trampa de la cátedra: *"No todo va en AH"*. Los datos pueden ir en AL, DL, DX, BX, CX, DS:DX, etc.
- **Registros** — AX, BX, CX y DX son de 16 bits y se parten en dos mitades de 8 (AH/AL, BH/BL, CH/CL, DH/DL); H es la mitad alta y L la baja. DS y ES son **registros de segmento**. SI, DI, BP y SP son de índice y punteros.
    - **DS:DX** — una sola dirección partida en dos: **segmento** (en qué bloque) : **desplazamiento** (a cuántos bytes del inicio). En un .COM con `ORG 100h` DS ya viene cargado; en un .EXE hay que cargarlo pasando por AX (`MOV AX, @data` / `MOV DS, AX`), porque un registro de segmento no admite un valor inmediato.
- **INT 21h (DOS), la más importante**
    - **01h** — lee un carácter CON eco → AL.
    - **02h** — muestra UN carácter → el carácter en **DL**.
    - **07h** — lee sin eco → AL. **08h** — lee sin eco y controla Ctrl-Break → AL.
    - **09h** — muestra una CADENA → dirección en **DS:DX**, terminada en **$**.
    - **0Ah** — lee una cadena a un buffer → DS:DX (byte 0 = capacidad, byte 1 = leídos, desde el byte 2 los caracteres).
    - **0Bh** — ¿hay entrada? → AL = FFh sí, 00h no.
    - **2Ah** / **2Ch** — fecha (CX año, DH mes, DL día) / hora (CH:CL:DH).
    - **3Ch–42h** — archivos: crear, abrir, cerrar, leer, escribir, borrar, seek (BX handle, CX cantidad, DS:DX buffer).
    - **4Ch** — termina el proceso → AL = código de retorno (o `MOV AX, 4C00h`).
- **INT 10h (video)** — **02h** posiciona el cursor (DH fila, DL columna, BH página); **09h** carácter con atributo (AL, BL, CX), no avanza el cursor; **0Ah** carácter sin atributo nuevo, no avanza; **0Eh** carácter y **avanza**; **13h** cadena completa (ES:BP, CX, DH/DL, BL).
    - 📌 La cartilla marca que 09h, 0Ah, 0Eh y 13h son las más fáciles de confundir.
- **INT 16h (teclado del BIOS)** — **00h** lee una tecla y **espera** (AL = ASCII, AH = scan code); **01h** consulta si hay tecla **sin bloquear** (ZF = 0 hay, ZF = 1 no hay); **02h** estado de Shift, Ctrl, Alt, Caps Lock y Num Lock en AL.
    - 🔗 00h **bloquea**; 01h **sondea** (polling): es el mismo par interrupción/espera ocupada del tema 3 (→ U1: Tema 3 — Las tres formas de hacer E/S).
- **INT 17h (impresora)** — 00h imprime un carácter (AL, DX = 0 para LPT1, 1 para LPT2); 01h inicializa; 02h devuelve el estado en AH.

- 🔢 **Mostrar una cadena** (el ejercicio del final práctica)
    1. Es una CADENA, no un carácter → INT 21h, AH = 09h.
    2. La 09h pide la dirección en DS:DX y la cadena terminada en `$`.
    3. `MOV DX, MSG` carga el desplazamiento.
    4. `MOV AH, 09h` elige la función.
    5. `INT 21h` ejecuta: DOS recorre desde DS:DX hasta el `$`.
    6. `RET` vuelve a DOS (en un .EXE: `MOV AX, 4C00h` + `INT 21h`).
- 📌 **Los 4 errores típicos que marca la cátedra**: usar **DL** en lugar de DX (DL es para un carácter, función 02h); olvidar el **$** (DOS muestra basura hasta encontrar un $ por casualidad); **no cargar DS**; usar **AH = 02h** para una cadena (muestra un solo carácter).
- 🛎️ **INT se parece a un CALL**: apila la dirección de retorno y el PSW, salta a la rutina y la rutina termina con **IRET**, que desapila todo. Es un CALL con control de seguridad. 🔗 Es el antecedente de TRAP (→ U1: Tema 6 — Llamadas al sistema).
- El código se ejecuta **de arriba abajo**. Se explica empezando por el INT porque ésa es la acción; las líneas de arriba son la preparación.

Tema 6 — Llamadas al sistema

📖 cap. 1.6, págs. 49–62; apunte Comandos_Linux-SO (1).pdf

🗒️ Repaso U1, pág. 21 — "Tema 6 — Llamadas al sistema" (resumen condensado: pág. 46)

- **Llamada al sistema** — la interfaz que permite a los programas de usuario pedir servicios al SO. Es la **frontera exacta** entre modo usuario y modo kernel.
- **TRAP** — instrucción que cambia a modo kernel y salta a una dirección fija del núcleo.
    - ⚖️ **TRAP contra CALL**: los dos guardan la dirección de retorno. El CALL salta a **cualquier** dirección y **no** cambia el modo; el TRAP salta a una dirección **FIJA** y **pasa a modo kernel**. Esas dos diferencias son toda la seguridad.
    - 🛎️ Un CALL te mueve dentro de tu casa; un TRAP te lleva a la oficina del gobierno, y solo por la puerta principal.
- **CALL, RET y la pila** — el procedimiento no sabe quién lo llamó: el CALL **apila** la dirección de la línea siguiente y salta; el RET **desapila** esa dirección y vuelve. Las llamadas anidadas se desapilan en orden inverso (LIFO). 🔗 Por eso cada hilo necesita su propia pila (→ U2: La pila).
- 🔢 **Los once pasos de `read(fd, bufer, nbytes)`**
    - a 3. El programa apila los parámetros en **orden inverso** (nbytes, &bufer, fd). El primero y el tercero van por **valor**; el segundo por **referencia** (la dirección del buffer).
    - Llama al **procedimiento de biblioteca** `read` (un CALL común).
    - El procedimiento de biblioteca (en ensamblador) pone el **número de la llamada al sistema** en el registro donde el SO lo espera.
    - Ejecuta **TRAP**: pasa a modo kernel y salta a una dirección fija.
    - El kernel usa el número como índice en una **tabla de apuntadores a manejadores**.
    - Se ejecuta el **manejador** de la llamada, que hace la lectura real.
    - El control vuelve al procedimiento de biblioteca, a la instrucción siguiente al TRAP.
    - El procedimiento de biblioteca retorna al programa.
    - El programa **limpia la pila** (incrementa el SP para sacar los tres parámetros).
    - 📌 **La llamada puede bloquear al proceso**: si se lee del teclado y no hay nada, el proceso se bloquea, el SO corre otro, y los pasos 9 a 11 se completan cuando llega la entrada. 🔗 Transición ① del diagrama de estados (→ U2: Tema 2 — Estados del proceso y el BCP).
    - **Valor de retorno** — la cantidad de bytes leídos (puede ser menor que nbytes si se llegó al fin del archivo). Si falla devuelve **–1** y el código de error queda en la variable global **errno**.
- **Llamadas POSIX principales**

    - **Procesos** — `fork` (crea un hijo idéntico) 🔑 (→ U2: Tema 1 — Qué es un proceso); `execve` (reemplaza la imagen); `exit` (termina); `waitpid` (espera a un hijo); `kill` (envía una señal).
    - **Archivos** — `open`, `read`, `write`, `close`, `lseek` (mueve el puntero), `stat` (información del archivo).
    - **Directorios y varias** — `mkdir`/`rmdir`, `link`/`unlink`, `mount`/`umount`, `chdir`, `chmod`, `time` (segundos desde el 1/1/1970).
    - **? Buffer** — una zona de memoria donde se copian los datos que se leen o se van a escribir.
    - **? `close(fd)`** — libera el descriptor: el número deja de servir. 🔑 Con hilos, lo pierden todos los hilos del proceso (→ U2: Tema 3 — Hilos: el modelo clásico).
- **POSIX contra Win32** — en POSIX la biblioteca y las llamadas al sistema van casi uno a uno. La **API Win32** tiene miles de funciones desacopladas a propósito de las llamadas reales, para que Microsoft pueda cambiar el kernel sin romper los programas.
- **strace** — muestra las llamadas al sistema que hace un programa.
    - 📌 Sintaxis exacta que piden: **`strace -o salida.txt ls -la`**. Distractores habituales: `>`, `|`, `-s`.
    - Qué aparece: `openat()` (abre archivos o el directorio "."), `read()`, `getdents64()` (lee las entradas de un directorio), `newfstatat()`/`statx()` (permisos, tamaño, dueño, fechas), `write()` (escribe la salida en la terminal), `close()`.
- 🔢 **Por qué conviene un buffer grande** (archivo de 8 KB)
    - 8 KB = 8192 bytes.
    - Con buffer de 512 bytes: 8192 / 512 = **16** llamadas a `read()`.
    - Con buffer de 4096 bytes: 8192 / 4096 = **2** llamadas.
    - Se leen **los mismos datos**, pero se cruza la frontera usuario → kernel 16 veces contra 2.
    - ⚠ **Un buffer más grande NO reduce la lectura de disco: reduce las llamadas al sistema.** (Error del cuestionario, parte B-e, 0/2.)

Tema 7 — Historia por generaciones

📖 cap. 1.2, págs. 7–19

Repaso U1, pág. 25 — "Tema 7 — Historia por generaciones" (resumen condensado: pág. 48)

- 💡 **Las fechas no importan; se toma por qué apareció cada invento.** La historia es una cadena de **agujeros de tiempo muerto** que se van tapando, porque las máquinas eran carísimas.
- **Agujero 1 (1945–1955, tubos al vacío): la máquina espera al humano** — se programaba cableando tableros; no había lenguajes ni SO. A principios de los 50, tarjetas perforadas. Antecedente: Babbage y su máquina analítica; Ada Lovelace, primera programadora.
- **Agujero 2 (1955–1965, transistores): la máquina espera al operador** — solución: **procesamiento por lotes**. Se juntan trabajos en una cinta (la 1401, barata, lee tarjetas) y la 7094 (cara) los ejecuta uno atrás del otro; después la 1401 imprime.
    - **Tarjetas de control** — `$JOB`, `$FORTRAN`, `$LOAD`, `$RUN`, `$END`: las precursoras de los shells. SO típicos: **FMS** e **IBSYS**.
- **Agujero 3 (1965–1980, circuitos integrados): la CPU espera al disco** — en datos comerciales la CPU esperaba E/S el 80 a 90 % del tiempo. Solución: **multiprogramación** (varios trabajos en memoria a la vez; cuando uno espera el disco, la CPU pasa a otro). 🔑 El modelo $1 - p^n$ (→ U1: Modelo de la multiprogramación).
    - ⚖ **Lotes contra multiprogramación**: lotes = **uno** en memoria por vez, en fila, y tapa el agujero **entre** trabajos; multiprogramación = **varios** en memoria a la vez, y tapa el agujero **dentro** de un trabajo. No se reemplazan: se suman.
    - 📌 **La multiprogramación exige hardware de protección** (si no, un trabajo pisa la memoria de otro). Recién lo trajo la tercera generación (IBM 360). 🔑 (→ U1: Modo kernel y modo usuario).
    - **IBM System/360** — primera familia compatible en software. Su OS/360 era tan enorme que corregir errores introducía otros.

    - **Spooling** — leer los trabajos al disco apenas llegan, para cargar otro apenas se libera una partición.
    - **Tiempo compartido** — cada usuario con su terminal; si de 20 conectados 17 están pensando, la CPU atiende por turnos a los 3 que piden. Primer sistema: **CTSS** (MIT). 🔗 Es el origen de Round Robin y de la planificación interactiva (→ U2: 8.3 — Round Robin).
    - **MULTICS** — "poder de cómputo como el servicio eléctrico". Primer sistema con memoria virtual y ligado dinámico.
    - **UNIX** — Ken Thompson, en una PDP-7. Se partió en System V y BSD → el IEEE creó **POSIX**. Después **MINIX** (Tanenbaum) y **Linux** (Torvalds).
- **Agujero 4: el usuario espera el resultado** — con lotes, el resultado llega horas después. Solución: tiempo compartido.
- **Agujero 5 (1980 a hoy, PC): ya no hay agujero** — la máquina es barata y el objetivo pasa a ser la comodidad de la persona. **CP/M** (Kildall), **MS-DOS** (Gates compra DOS por unos 75.000 dólares y lo vende incluido con el hardware), la **GUI** (Engelbart → Xerox PARC → Jobs: Lisa fracasa, Macintosh triunfa), **Windows** (entorno sobre DOS hasta Windows 95; NT es una reescritura de cero en 32 bits).
- 🌤 **"La ontogenia recapitula la filogenia"** — ideas descartadas que vuelven en otro contexto: ensamblador, monoprogramación y directorios de un solo nivel reaparecen en los sistemas embebidos y las tarjetas inteligentes.
- **Cuadro**: 1 máquina espera al programador → operadores; 2 espera al operador → lotes; 3 CPU espera al disco → multiprogramación + spooling; 4 usuario espera el resultado → tiempo compartido; 5 la máquina es barata → PC y GUI.

Tema 8 — Tipos de sistemas operativos

📖 cap. 1.4, págs. 33–37

🗐 Repaso U1, pág. 30 — "Tema 8 — Tipos de sistemas operativos" (resumen condensado: pág. 51)

- **Los nueve tipos, del más grande al más chico**
    - **De mainframe** — lo que los distingue es la **capacidad de E/S**. Tres servicios: lotes, **procesamiento de transacciones** (muchos pedidos chicos) y tiempo compartido. Ej.: OS/390.
    - **De servidores** — varios usuarios por red, compartiendo hardware y software. Ej.: Solaris, FreeBSD, Linux, Windows Server.
    - **De multiprocesadores** — varias CPU en un sistema. 🔗 Desarrollado en la U2, tema 10 (→ U2: Tema 10 — Sistemas de múltiples procesadores).
    - **De computadoras personales** — buen soporte a un solo usuario.
    - **De bolsillo** — celulares y PDA (Symbian, Palm OS).
    - **Integrados** — controlan dispositivos que no se consideran computadoras; todo el software en ROM (QNX, VxWorks).
    - **De nodos sensores** — minúsculos, a batería, por radio (TinyOS).
    - **De tiempo real** — el tiempo es el parámetro clave.
    - **De tarjetas inteligentes** — un chip en una tarjeta, a veces con una sola función.
- **Tiempo real duro contra suave** — 📌 se toma.
    - **Duro** — la acción **debe** ocurrir en el instante exacto, sin excepción; garantías absolutas. Ej.: robot soldador, aeronáutica, control de procesos, aplicaciones militares.
    - **Suave** — es aceptable fallar muy ocasionalmente. Ej.: audio digital, multimedia, teléfonos digitales.
    - 🔗 En la U2 aparece la condición de planificabilidad $\Sigma\ C_i/P_i \leq 1$ (→ U2: 8.4 — Planificación de tiempo real).
    - **?** Sí: el "tiempo real duro" de la U1 es el mismo concepto que el "duro" de la planificación de tiempo real de la U2.
- ⚖ **Integrado contra de bolsillo**: en el de bolsillo instalás apps de terceros → puede correr software no confiable → necesita protección. En el integrado todo está en ROM → **certeza de que nunca correrá**

software no confiable → no necesita protección y el diseño se simplifica.

Tema 9 — Estructuras de los sistemas operativos

📖 cap. 1.7, págs. 62–72

🗐 Repaso U1, pág. 32 — "Tema 9 — Estructuras de los sistemas operativos" (resumen condensado: pág. 52)

- 💡 **Las cinco estructuras responden a una sola pregunta: ¿quién corre en modo kernel?** Más cosas en el kernel = más rápido y más frágil; menos cosas = más seguro y más lento. Es el intercambio **rendimiento contra confiabilidad**.
- **1. Monolítica** — un solo binario enorme en modo kernel; cualquier procedimiento llama a cualquier otro. Por dentro: programa principal → procedimientos de servicio → procedimientos utilitarios. Rapidísima; un error en cualquier módulo tira todo. Es la más común (Linux).
  - **Módulos cargables** — Linux "hace trampa": pedazos de código (típicamente drivers) que se insertan y se quitan del kernel en caliente.
- **2. Por capas** — el sistema **THE** (6 capas: 0 asignación del procesador y multiprogramación, 1 memoria, 2 comunicación operador-proceso, 3 E/S, 4 programas de usuario, 5 el operador). **MULTICS** lo generalizó con **anillos** concéntricos.
  - ⚠ **Las capas NO dan tolerancia a fallas**: todo sigue en modo kernel, y un driver que falla tumba el sistema igual que en la monolítica. Lo que mejora es la **claridad para depurar**. (Error de la clase y del cuestionario, parte B-b, 0/2.)
  - 💡 **Estar ordenado no es estar protegido.** Frase para memorizar: *en capas todo sigue en modo kernel.* La tolerancia a fallas la da el microkernel, que saca los drivers afuera.
- **3. Microkernel** — solo el microkernel corre en modo kernel; los drivers y servidores son procesos de usuario. Un driver con errores no tumba el sistema: el SO lo detecta, lo mata y lo reinicia (**MINIX 3**).
  - Los drivers son la causa más frecuente de caídas (código de terceros).
  - **Mecanismo en el kernel, política en el espacio de usuario**. 🔗 La misma regla en planificación (→ U2: 8.5 — Política contra mecanismo).
  - Costo: lo que en el monolítico es un CALL, acá es un **mensaje** entre procesos.
  - Un driver en modo usuario toca el hardware **pidiéndoselo al microkernel** por una llamada al sistema, y solo para los puertos de su dispositivo.
- **4. Cliente-servidor** — variación del microkernel: servidores y clientes comunicados por **paso de mensajes**; el cliente no sabe si el servidor está en la misma máquina o en la red. Base de los sistemas distribuidos (U8). 🔗 (→ U2: Tema 7.4b — Paso de mensajes y barreras).
  - ⚠ **No confundir la estructura del SO con la arquitectura de una aplicación** (tu app que le manda SQL a un motor). Y no elegir cliente-servidor para algo sensible a la latencia: es el más lento.
- **5. Máquinas virtuales** — el monitor separa multiprogramación y máquina extendida: ofrece **copias exactas del hardware**, con modo kernel, modo usuario e interrupciones (VM/370 de IBM).
  - **Hipervisor tipo 1** — corre **directamente sobre el hardware** (VM/370).
  - **Hipervisor tipo 2** — corre como **programa de usuario sobre un SO anfitrión** (VirtualBox sobre Windows).
  - **? El SO de una PC común no es "tipo 1"**: no es un hipervisor. Tipo 1 y tipo 2 clasifican **hipervisores**, no sistemas operativos.
  - **JVM** — una arquitectura definida por software, con un intérprete de **código byte**: por eso un `.class` corre en cualquier máquina con JVM.
  - **? La JVM no es como un `venv` de Python**: el venv solo aísla paquetes; la JVM ejecuta un código máquina propio (el código byte).
  - **Exokernel** — lo opuesto: en vez de copiar la máquina, le da a cada usuario un **subconjunto de los recursos reales**, y se ahorra la capa de traducción.

- **Comparación final** — monolítica: todo el SO en kernel, rendimiento, un error tumba todo (Linux). Capas: todo en kernel ordenado, claridad (THE, MULTICS). Microkernel: solo el microkernel, confiabilidad, lento (MINIX 3, QNX). Cliente-servidor: microkernel + mensajes, se extiende a la red, latencia. Máquinas virtuales: el hipervisor, aislamiento total, sobrecarga.
- 🌤️ **Para elegir: ¿qué me duele más perder?** Rendimiento → monolítica; confiabilidad → microkernel. En tiempo real duro casi siempre microkernel (QNX en aviónica, medicina y autos).
- 🩹 **Es el mismo intercambio que hilos contra procesos**: Chrome eligió "procesos" como quien elige microkernel (→ U2: Tema 3 — Hilos: el modelo clásico).

Lo que más se toma de la U1

🗐 Repaso U1, pág. 37 — "Cierre de la Unidad 1 › Lo que más se toma de esta unidad" (resumen condensado: pág. 55)

- **Casi seguro que cae**
  - `strace -o salida.txt ls -la` (→ U1: strace).
  - Comandos de las cartillas: `ls -l /dev | grep '^c'` (dispositivos de carácter; `^b` de bloque), `pgrep firefox` o `ps -ef | grep firefox` (PID), `vmstat` (columna `id` = % de CPU inactiva), `free -h` o `cat /proc/meminfo` (RAM), `lsblk -f` (dispositivos de bloque y espacio), `cat /etc/fstab` (tamaño de bloque de una partición ext4), `kill -STOP PID` / `kill -CONT PID` (suspender y reanudar), `iostat` (E/S).
  - INT 21h: completar el código para mostrar una cadena (AH = 09h, cadena con $, DS:DX) (→ U1: INT 21h (DOS), la más importante).
- **Aparece seguido** — las tres formas de E/S y cuál conviene; modo kernel contra usuario y la instrucción privilegiada; lotes contra multiprogramación (V/F justificado); tiempo real duro contra suave.
- **Suele ser desarrollo** — las cinco estructuras y su intercambio; la historia y por qué aparece cada invento.

U2 — Procesos, hilos, concurrencia y planificación

📖 Tanenbaum, cap. 2 completo (págs. 83–168) + cap. 8.1 (págs. 526–548) + apunte de la cátedra

🗐 Repaso U2, 168 págs. — mapa de la unidad en la pág. 1; cierre en la pág. 167

La unidad es una sola historia en tres actos. **Acto 1 — ¿qué ejecuta el sistema?**: proceso → estados y BCP → hilos → implementación de hilos. **Acto 2 — ¿cómo evitamos que se pisen?**: condiciones de carrera → exclusión mutua con espera ocupada → sin espera ocupada (semáforos, mutexes, monitores, mensajes). **Acto 3 — ¿a quién le toca?**: planificación → problemas clásicos → multiprocesadores.

Tema 1 — Qué es un proceso

📖 cap. 2.1.1 a 2.1.4, págs. 84–90

🗐 Repaso U2, pág. 1 — "Tema 1 — Qué es un proceso"

- **Proceso** — *"una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables"*.
  - 🌤️ **La torta del libro**: la receta es el **programa**, los ingredientes son los **datos**, el informático cocinando es el **proceso** y él mismo es la **CPU**. Si el hijo se corta, anota en qué paso estaba (el **estado**), sigue otra receta (primeros auxilios) y después vuelve a la torta donde la dejó.
  - ⚖️ **Programa contra proceso**: el programa es **pasivo** (un archivo en el disco, no cambia); el proceso es **activo** (tiene estado y el estado cambia). **Varios procesos pueden correr el mismo programa.**
  - **?** Sí: lo que ocupa RAM es el **proceso**; el **programa** está en el disco.
  - 📌 Tres ventanas del mismo editor = **1 programa y 3 procesos**. Comparten el **código**; cada uno tiene su PID, su espacio de direcciones, sus variables, su PC, su pila, sus registros y sus archivos abiertos.

- **Pseudoparalelismo** — *"en cualquier instante la CPU está ejecutando sólo un proceso"*; conmuta cada décimas o centésimas de milisegundo y eso da la **ilusión** de paralelismo.
  - ⚖️ **Pseudoparalelismo contra paralelismo real**: la ilusión producida por conmutación rápida sobre **una** CPU, contra dos o más CPU ejecutando **en el mismo instante**.
  - 📌 50 procesos en un solo núcleo es un sistema **concurrente**, no **paralelo**. 🔗 El paralelismo real es el tema 10 (→ U2: Tema 10 — Sistemas de múltiples procesadores).
- **Los 4 eventos que crean procesos**
  - El arranque del sistema.
  - Una llamada al sistema de creación hecha por otro proceso (la más común).
  - Una petición del usuario (un comando, un doble clic).
  - El inicio de un trabajo por lotes.
  - **Demonio** — proceso de segundo plano que arranca con el sistema (correo, web, impresión).
  - **?** El evento 2 quiere decir que un proceso que ya está corriendo ejecuta `fork` (que es una llamada al sistema, o sea un TRAP) 🔗 (→ U1: TRAP).
- **fork y execve**
  - `fork()` — **duplica** el proceso: el hijo es un clon con la **misma imagen de memoria, las mismas cadenas de entorno y los mismos archivos abiertos**.
  - `execve()` — **reemplaza** la imagen del proceso por otro programa.
  - ⚖️ **UNIX contra Windows**: en UNIX son **dos llamadas separadas** (fork + execve); en Windows, `CreateProcess` hace las dos cosas en **una sola llamada** con más de 10 parámetros.
  - 🔔 **El valor de retorno de fork**: al **padre** le devuelve el **PID del hijo** (> 0); al **hijo** le devuelve **0**; si falla devuelve **–1**. El código es idéntico en los dos: lo único que los distingue es ese número.
  - 🔢 **fork + execve, y la redirección del shell** (`ls -la > salida.txt`)
    - El shell hace `fork()` y se crea el hijo.
    - En el hijo: `close(1)` cierra la salida estándar.
    - `open("salida.txt")` → el SO le asigna el descriptor libre más bajo, que es el **1**.
    - `execve("ls")` → `ls` escribe en el fd 1 como siempre, pero el fd 1 ahora es el archivo.
    - Por eso fork y exec están separados: entre los dos, el hijo puede acomodar sus descriptores.
  - **? Después de fork, ¿son espacios de memoria distintos?** Sí: el hijo tiene **su propio** espacio de direcciones, que empieza como copia del padre. Lo que escribe uno no lo ve el otro (en la práctica se comparte el código de solo lectura o se copia recién al escribir).
  - 🔔 **n fork seguidos sin `if` = 2^n procesos.** Cada fork **duplica todo lo que hay**, porque el hijo arranca justo después del fork que lo creó y también ejecuta los siguientes.
    - ⚠️ **Contesté 4 para tres fork; son 8** (7 hijos + el original). El fork no suma uno: multiplica por dos.
    - 📌 Con un `if (fork() == 0) { fork(); }` ya no es 2^n: hay que seguir el árbol rama por rama.
- **Las 4 razones de terminación**
  - **Voluntarias** — 1 salida normal (`exit`, `ExitProcess`); 2 salida por error (por ejemplo, compilar un archivo que no existe).
  - **Involuntarias** — 3 error fatal (instrucción ilegal, división por cero, memoria inexistente); 4 eliminado por otro (`kill`, `TerminateProcess`).
- **Jerarquía de procesos** — en **UNIX** hay un **árbol** real con **init** como raíz (init hace un fork por cada terminal; un proceso no puede desheredar a sus hijos; `pstree` lo dibuja). En **Windows no hay jerarquía**: el padre recibe un **token** para controlar al hijo, pero puede pasárselo a otro.

Tema 2 — Estados del proceso y el BCP

📖 cap. 2.1.5 y 2.1.6, págs. 90–93 (fig. 2-2 y fig. 2-5)

🗐 Repaso U2, pág. 5 — "Tema 2 — Estados del proceso y el BCP"

- **Los tres estados**
  - **En ejecución** — está usando la CPU en este instante.
  - **Listo** (preparado) — es ejecutable; solo le falta la CPU.
  - **Bloqueado** — no puede continuar hasta que ocurra un evento externo, **aunque la CPU esté libre**.
  - 🔔 **La guardia del hospital**: en el consultorio (ejecución), en la sala de espera (listo), en el laboratorio esperando un análisis (bloqueado): aunque el médico esté libre, no lo pueden atender.
  - ⚖ **Listo contra bloqueado**: listo = *podría*, solo le falta el turno; bloqueado = *no puede* por lógica, le falta un dato.
- **Las 4 transiciones y quién provoca cada una**
  - **Ejecución → Bloqueado** — la provoca **el propio proceso**: pidió algo que no está.
  - **Ejecución → Listo** — la provoca **el planificador**: se acabó el quantum o llegó alguien mejor.
  - **Listo → Ejecución** — la provoca **el planificador**: le toca.
  - **Bloqueado → Listo** — la provoca **un evento externo**: llegó el dato (una interrupción).
  - 📌 **No existe Bloqueado → Ejecución.** Cuando llega el dato, el proceso pasa a LISTO y espera su turno: ④ **y después ③, nunca una sola transición.**
  - 📌 **Tampoco existe Listo → Bloqueado**: para bloquearse hay que *pedir* algo, y para pedir hay que estar ejecutándose.
  - 🔔 **Transición** = **un** solo cambio de estado (una flecha). En una secuencia hay **una transición menos** que estados visitados.
  - 🔔 El que se bloquea **actúa** (①); el que se desbloquea **es actuado** (④).
  - ⚠ **Dije que "el proceso bloqueado hace una interrupción"**. No: un proceso bloqueado no puede hacer nada. La interrupción la genera **el controlador del dispositivo** (hardware) (→ U1: Tratamiento de una interrupción por hardware).
  - ⚠ **Fin de quantum → LISTO, nunca BLOQUEADO.** Lo decide el planificador. Pasar a BLOQUEADO lo decide el proceso, pidiendo algo que no tiene. (Error que repetí en la U2 dos veces: en la inversión de prioridades y en la respuesta del 10.4.)
  - 📌 Un proceso de más prioridad que se desbloquea **no salta** directo a ejecución: el planificador aplica ② al actual y ③ al desbloqueado (dos transiciones), y en un sistema **no apropiativo** ni siquiera eso.
  - **? ¿Para qué se genera la interrupción si hay 6 procesos listos antes?** La interrupción no es para que lo atiendan ya: es para que el manejador lo pase de BLOQUEADO a LISTO (④). Después decide el planificador.
- 🔢 **Seguir un proceso que lee del disco**
  - P está EN EJECUCIÓN.
  - P hace `read()`; los datos hay que ir a buscarlos al disco.
  - ① El SO lo pasa a BLOQUEADO y le pide al driver que inicie la operación.
  - ③ El planificador elige a Q (que estaba LISTO). La CPU no queda ociosa: **eso es la multiprogramación** 🚀 (→ U1: Tema 7 — Historia por generaciones).
  - El disco termina y genera una **interrupción**: ④ el manejador pasa a P a LISTO. **P todavía no se ejecuta.**
  - ③ Cuando el planificador lo decide, P vuelve a EJECUCIÓN y sigue después del `read()`.
  - Desde adentro, P cree que el `read()` tardó todo eso; pero una parte fue **espera de disco** (bloqueado) y otra **espera de CPU** (listo).
- **BCP (bloque de control de proceso)** — cada entrada de la **tabla de procesos** (un arreglo con una entrada por proceso). Guarda todo lo necesario para reanudar al proceso como si nunca se hubiera detenido.
  - **Administración de procesos** — registros, contador de programa, PSW, apuntador de pila,

estado, prioridad, PID, PPID, tiempo de CPU usado, señales.
    - **Administración de memoria** — apuntadores a los segmentos de texto, datos y pila.
    - **Administración de archivos** — directorio raíz, directorio de trabajo, **descriptores de archivos**, UID y GID.
    - 🔗 Los tres primeros campos (registros, PC, PSW) son los de la U1: el SO tiene que conocerlos **para guardarlos y restaurarlos** (→ U1: La CPU y sus registros).
    - ⚖️ **PSW contra campo estado**: el PSW es un registro del hardware que se *guarda* en el BCP; el estado (ejecución/listo/bloqueado) es un dato del SO.
    - **? El BCP es del proceso**; cada hilo tiene además su propia entrada en una tabla de hilos con su PC, registros, pila y estado. El diagrama de estados vale igual para los hilos (→ U2: Tema 3 — Hilos: el modelo clásico).
    - **Descriptor de archivo (fd)** — un número, índice en la tabla de descriptores del proceso (el ticket del guardarropa). Los **0, 1 y 2** (entrada, salida y error estándar) están siempre abiertos. `ls -la /proc/PID/fd` lista esa tabla.
- 🔢 **Manejo de una interrupción y cambio de contexto** (fig. 2-5)
    - El **hardware** apila el contador de programa, el PSW, etc.
    - El **hardware** carga el nuevo contador de programa desde el **vector de interrupciones**.
    - Una rutina en **ensamblador** guarda los registros (en el BCP del proceso actual).
    - La rutina en ensamblador establece la nueva pila.
    - Se ejecuta el servicio de interrupciones en **C** (por lo general lee y guarda la entrada en un búfer).
    - El **planificador** decide qué proceso se ejecuta a continuación.
    - El procedimiento en C regresa al código en ensamblador.
    - La rutina en ensamblador **inicia el nuevo proceso actual** (carga sus registros y su mapa de memoria).
    - 📯 **"Esto es todo lo que hace el hardware"**: los pasos 1 y 2. Del 3 en adelante es software.
    - 📌 Guardar los registros y cambiar la pila **no se puede escribir en C**: por eso esa parte es ensamblador.
    - **El cambio de contexto no es gratis**: pasar a modo kernel, guardar y restaurar registros, cambiar el mapa de memoria y muchas veces **invalidar la caché**. 🔗 Es lo que hace que el tamaño del quantum sea un problema real (→ U2: 8.3 — Round Robin).
    - **? ⚖️ Planificador contra manejador de interrupciones**: el manejador *atiende el evento* (pasa a alguien a LISTO); el planificador *decide quién usa la CPU* después. Son partes distintas del SO, y los dos corren en modo kernel.
    - **? ⚖️ SO, CPU y planificador son cosas distintas**: la CPU es el hardware que ejecuta; el SO es el software que administra; el planificador es **una parte del SO** que elige el próximo proceso.
- **Comandos de la cátedra** — `ps -ef` (UID, PID, PPID, C, STIME, TTY, TIME, CMD); `pstree`; `top`/`htop` (columna S = estado); `pgrep firefox`; `cat /proc/PID/status` (nombre, estado, PID, PPID, memoria, cantidad de hilos: es **asomarse al BCP**); `ls -la /proc/PID/fd`; `kill -STOP PID` (suspende) y `kill -CONT PID` (reanuda).

Tema 3 — Hilos: el modelo clásico

📖 cap. 2.2.1 y 2.2.2, págs. 95–104

🗐 Repaso U2, pág. 9 — "Tema 3 — Hilos: el modelo clásico"

- 📯 **El restaurante**: una cocina (ingredientes, ollas, heladera) con tres cocineros adentro. Comparten la cocina entera, sin paredes, a propósito.
- 📯 **El proceso AGRUPA recursos; el hilo EJECUTA.** Al hilo también se lo llama **proceso ligero**.
- **La tabla que más se toma**
    - **Por proceso (compartido por todos sus hilos)** — espacio de direcciones, variables globales, archivos abiertos, procesos hijos, alarmas pendientes, señales y manejadores de señales,

información contable.
    - **Por hilo (propio de cada uno) — contador de programa, registros, pila y estado**. Solo 4 cosas.
    - 🔔 Lo propio del hilo es **"dónde voy en la ejecución"**; lo del proceso es **"con qué trabajo"**.
- **La pila** — zona de memoria **LIFO** (el último que entra es el primero que sale). Por cada procedimiento activo guarda un **marco de pila**: la **dirección de retorno**, los **parámetros** y las **variables locales**. El registro SP apunta al tope (→ U1: La CPU y sus registros).
    - Organización de la memoria de un proceso, de arriba abajo: **pila** (crece hacia abajo), espacio libre, **heap** (crece hacia arriba: malloc, new), **datos** (variables globales), **texto** (el código).
    - 📌 **Por qué cada hilo necesita su propia pila**: cada uno tiene un historial de llamadas distinto. Con una pila compartida, el `return` de un hilo desapilaría el marco del otro. 🧠 Es la pila del CALL/RET de la U1 (→ U1: Tema 6 — Llamadas al sistema).
- **No hay protección entre hilos** — un hilo puede leer, escribir o borrar la pila de otro, y es **intencional**: son del mismo usuario y cooperan. El precio: si uno corrompe memoria, **cae todo el proceso**.
    - 📌 `close(fd)` en un hilo: el descriptor es del proceso, así que **desaparece para todos**; si otro hilo estaba leyendo, su `read()` falla. Con procesos separados, cada uno tiene su tabla de descriptores.
    - 🔔 **Compartir ≠ coordinarse**: compartir lo hace el sistema gratis; coordinarse lo escribe el programador. Compartir sin coordinarse produce **condiciones de carrera** (→ U2: Tema 5 — Condiciones de carrera y regiones críticas).
- **Las 4 razones para usar hilos**
    1. Varias actividades a la vez y algunas se bloquean: hilos secuenciales simplifican la programación.
    2. Comparten el espacio de direcciones y los datos.
    3. Son más ligeros: se crean de 10 a 100 veces más rápido.
    4. Permiten traslapar cálculo y E/S, y con varias CPU dan paralelismo real.
- **Ejemplo del procesador de palabras** — un hilo interactúa con el usuario, otro reformatea en segundo plano, otro respalda a disco. Con procesos no compartirían el documento; con un solo hilo tendrías que esperar el reformateo.
- **Ejemplo del servidor web** — un **despachador** recibe pedidos y los pasa a **hilos trabajadores**, que comparten la caché de páginas.
    - **Las tres formas de construirlo — hilos**: paralelismo + llamadas con bloqueo (programación simple y buen rendimiento); **un solo hilo**: sin paralelismo + llamadas con bloqueo (simple pero lento); **máquina de estados finitos**: paralelismo + llamadas **sin** bloqueo (rápido pero muy difícil de programar).
    - 🔔 *Los hilos combinan paralelismo con llamadas al sistema con bloqueo.*
- **Llamadas de hilos** — `thread_create`, `thread_exit`, `thread_join` (espera a que otro termine), `thread_yield` (cede la CPU a otro hilo). Los hilos tienen los mismos estados que los procesos.
    - 🔔 **`thread_yield` existe porque dentro de un proceso no hay interrupciones de reloj**: los hilos de usuario tienen que cederse la CPU voluntariamente (→ U2: Tema 4 — Implementación de hilos).
- **Complicaciones** — `fork` en un proceso multihilo (¿el hijo hereda todos los hilos?); variables globales como `errno` que dos hilos pisan; bibliotecas no reentrantes; las **señales** van a procesos, no a hilos; varias pilas que el kernel no administra si no sabe que hay hilos.
- ⚖️ **Por qué Chrome usa un proceso por pestaña** — ① **aislamiento de fallas** (una pestaña que crashea no tira el navegador); ② **protección de memoria** (una pestaña maliciosa no puede leer la del homebanking). Paga más memoria a propósito.
    - 🧠 **Es el mismo intercambio que monolítico contra microkernel** (→ U1: Tema 9 — Estructuras de los sistemas operativos), y vuelve con memoria compartida contra paso de mensajes (→ U2: Tema 7.4b — Paso de mensajes y barreras).

Tema 4 — Implementación de hilos

📖 cap. 2.2.4 a 2.2.8, págs. 106–114

🗒 Repaso U2, pág. 17 — "Tema 4 — Implementación de hilos"

- 🔔 **La pregunta que define todo: ¿el kernel sabe que existen los hilos?**
- **A. Hilos en el espacio de usuario** — el kernel no sabe nada; cada proceso tiene su **tabla de hilos** y un **sistema en tiempo de ejecución** (una biblioteca) que los administra.
  - **Ventajas** — funcionan en un SO que no tiene hilos; conmutar es **rapidísimo** (sin TRAP, sin cambio de contexto, sin vaciar la caché); cada proceso puede tener **su propio algoritmo de planificación**; escalan mejor.
  - **Desventajas**
    - **Una llamada al sistema bloqueante bloquea a TODO el proceso**: el kernel solo bloquea lo que conoce, y conoce el proceso. Parche: **envolturas** que preguntan antes con `select`.
    - Un **fallo de página** también bloquea al proceso entero.
    - **No hay interrupciones de reloj dentro del proceso**: un hilo que no llama a `thread_yield` no cede nunca.
    - 📌 **La paradoja**: los programadores quieren hilos justamente en las aplicaciones que hacen muchas llamadas bloqueantes, que es donde los hilos de usuario funcionan peor.
  - **? Sí tienen planificador**: es el del sistema en tiempo de ejecución, dentro del proceso. Lo que no tienen es **reloj** que lo dispare.
  - ⚠ **Atribuí a "otro espacio de memoria" que un hilo egoísta no afecte a otros procesos.** La razón correcta: **el kernel sí tiene interrupciones de reloj** (para los procesos).
- **B. Hilos en el kernel** — el kernel tiene una **tabla de hilos** global y los conoce uno por uno.
  - Si un hilo se bloquea, el kernel corre **otro hilo** (del mismo proceso o de otro). Un fallo de página tampoco detiene al resto.
  - **? No hacen falta llamadas sin bloqueo ni envolturas**: como el kernel ve cada hilo, una llamada bloqueante bloquea solo a ese hilo.
  - Costo: crear, destruir y conmutar son **llamadas al sistema** (más caras). Mitigación: **reciclar hilos**.
  - Problemas que no resuelve: qué hacer con los hilos en un `fork` y quién atiende una **señal**.
- **Comparación (la tabla que cae)** — usuario: el kernel no los ve, conmutación rapidísima, una llamada bloqueante bloquea todo, sin reloj interno, planificación propia. Kernel: el kernel los ve, conmutación cara, solo se bloquea ese hilo, el kernel los reparte.
- **C. Implementación híbrida (M:N)** — varios hilos de usuario multiplexados sobre algunos hilos de kernel.
  - ⚠ **Mi objeción, correcta**: con 1 hilo de kernel es igual a hilos de usuario puros (**N:1**), y con tantos hilos de kernel como de usuario es igual a hilos de kernel puros (**1:1**). Esos son los **extremos**; la híbrida es **M:N**, el punto intermedio.
  - 🔔 **Hilos de usuario = cuántas tareas lógicas querés (miles); hilos de kernel = cuánto paralelismo real podés tener (≈ cantidad de núcleos).** Ejemplo: 1000 hilos de usuario sobre 4 de kernel.
  - 🔔 **Lo que se bloquea no es un grupo de hilos: es un ASIENTO.** Si un hilo de usuario se bloquea en una llamada, se pierde **un hilo de kernel de N** (capacidad de ejecución); ningún otro hilo de usuario queda atrapado. Los asientos se asignan dinámicamente.
  - El hilo de kernel no "encapsula" hilos de usuario: es un **vehículo** en el que se montan de a uno.
- **Activaciones del planificador** (Anderson) — imitan a los hilos de kernel con el rendimiento de los de usuario: el kernel asigna **procesadores virtuales** a cada proceso y, cuando un hilo se bloquea, avisa al sistema en tiempo de ejecución con una **llamada ascendente (upcall)**. Objeción: **viola el modelo de capas** (una capa inferior llama a una superior).
- **Hilos emergentes** — la llegada de un **mensaje** (por ejemplo, un pedido por la red) crea un hilo nuevo para atenderlo. Como no tienen historia que restaurar, se crean rapidísimo. Si corren en el kernel son más rápidos, pero un error es más peligroso.
  - **?** Un pedido HTTP con un JSON a un endpoint es un buen ejemplo del "mensaje" que dispara un

hilo emergente.

- 🔗 **La planificación de estos dos tipos de hilos** es el tema 8.6 (→ U2: 8.6 — Planificación de hilos).

Tema 5 — Condiciones de carrera y regiones críticas

📖 cap. 2.3.1 y 2.3.2, págs. 117–120 (figs. 2-21 y 2-22)

🗒 Repaso U2, pág. 25 — "Tema 5 — Condiciones de carrera y regiones críticas"

- **Las 3 cuestiones de la comunicación entre procesos (IPC)** — 1 cómo pasarse información; 2 que no se interpongan (que dos no tomen lo mismo a la vez); 3 el orden correcto cuando hay dependencias. Las dos últimas valen igual para los hilos.
- **Condición de carrera** — dos o más procesos leen o escriben datos compartidos y **el resultado final depende del orden** en que se ejecutaron.
  - Son difíciles de depurar porque **casi siempre funciona**: el error aparece solo con una intercalación precisa.
  - 🔢 **Los dos cajeros** (la intuición de la clase) — una cuenta conjunta con $1000; dos personas sacan $600 al mismo tiempo en cajeros distintos.
    - El cajero A lee el saldo: 1000.
    - El cajero B lee el saldo: 1000.
    - Los dos verifican "¿alcanza para 600?" → sí.
    - A entrega 600 y escribe saldo 400; B entrega 600 y escribe saldo 400.
    - **Se entregaron $1200 de una cuenta que tenía $1000**, y el saldo final (400) es coherente con una sola de las dos operaciones. Nadie detectó un error.
  - 🔔 **El patrón de toda condición de carrera: leer → modificar → escribir, interrumpido en el medio.**
- 🔢 **El spooler de impresión** (el ejemplo del libro)
  1. El directorio de spooler tiene ranuras numeradas; la variable compartida `in` indica la próxima libre (vale 7).
  2. A lee `in` = 7 y lo guarda en su variable local.
  3. **Interrupción de reloj**: el planificador pasa a B.
  4. B lee `in` = 7, pone su archivo en la ranura 7 y deja `in` = 8.
  5. Vuelve A: tiene guardado 7, pone su archivo en la ranura 7 (**pisa el de B**) y deja `in` = 8.
  6. El directorio quedó consistente, así que **nadie se entera**: el archivo de B no se imprime nunca.
- **Región crítica (sección crítica)** — la parte del programa donde un proceso accede a memoria compartida, archivos compartidos o cualquier recurso que pueda producir una condición de carrera.
- **Exclusión mutua** — la regla de que si uno está usando el recurso compartido, los demás no pueden usarlo a la vez.
  - **? ⚖ "¿La región crítica es la habitación y la exclusión mutua el candado?"** Casi. La región crítica es el **código** que usa el recurso (estar adentro del baño); la exclusión mutua es la **propiedad** de que haya uno por vez; el **candado** es el **mecanismo** que la implementa (temas 6 y 7).
  - **? Recursos compartidos** — una variable global, un buffer, un archivo, una tabla del kernel, una impresora, una base de datos.
- **Las 4 condiciones de una buena solución** (textuales, se toman)
  - No puede haber **dos procesos a la vez** dentro de sus regiones críticas.
  - No pueden hacerse **suposiciones sobre las velocidades ni sobre la cantidad de CPU**.
  - Ningún proceso que se ejecute **fuera** de su región crítica puede **bloquear** a otros procesos.
  - Ningún proceso tiene que **esperar para siempre** para entrar a su región crítica.
  - 🔔 **Para saber cuál viola un intento** (truco para el 6): preguntate quién bloquea a quién y si el que

bloquea estaba usando la región crítica. **Si no la estaba usando, es la condición 3.**
    - **? Si al que está en la región crítica se le acaba el quantum**: pasa a LISTO **sin salir** de la región; los que quieren entrar siguen sin poder (esperan o se bloquean), y los que no necesitan el recurso se ejecutan normalmente. **Perder la CPU no es salir de la región crítica.** 🔑 Es la semilla de la inversión de prioridades (→ U2: Inversión de prioridades).

Tema 6 — Exclusión mutua con espera ocupada

📖 cap. 2.3.3, págs. 120–125 (figs. 2-23, 2-24, 2-25 y 2-26)

🗐 Repaso U2, pág. 29 — "Tema 6 — Exclusión mutua con espera ocupada"

- **Espera ocupada** — probar una variable en un ciclo continuo hasta que tenga cierto valor. **Gasta CPU sin avanzar.** Al candado que usa espera ocupada se lo llama **candado de giro (spin lock)**.
    - 🔑 Es la misma idea de la E/S programada de la U1: sondear sin parar (→ U1: 1. E/S programada (espera ocupada, polling)).
- ✗ **Solución 1 — Deshabilitar interrupciones** — cada proceso las deshabilita al entrar y las rehabilita al salir: sin interrupciones de reloj no hay conmutación.
    - Falla 1: es imprudente darle ese poder a un proceso de usuario (si nunca las rehabilita, se termina el sistema).
    - Falla 2: en un **multiprocesador** solo afecta a **la CPU que las deshabilitó**; las otras siguen entrando. Viola la **condición 2** (suposiciones sobre la cantidad de CPU).
    - Sí es útil **dentro del kernel**, por unas pocas instrucciones.
    - 🔔 **El reloj late siempre**: deshabilitar interrupciones no lo detiene, solo hace que la CPU no lo escuche. Así es como el SO le saca la CPU a un proceso (la interrupción de reloj llama al planificador).
    - 🔔 **El quantum es un contador que se descuenta en cada tic.** Con un reloj de 1 ms y un quantum de 50 ms hay 50 interrupciones de reloj, y **49 no cambian nada**: solo la última dispara la transición ②. Interrupción de reloj ≠ fin de quantum.
- ✗ **Solución 2 — Variables de candado** — una variable compartida en 0 (libre) o 1 (ocupado). Falla porque **leer y escribir son dos pasos**: dos procesos pueden leer 0 antes de que alguno escriba 1. Es el mismo error del spooler. Viola la **condición 1**.
- ✗ **Solución 3 — Alternancia estricta** — la variable `turno` dice a quién le toca; cada uno espera su turno y al salir se lo pasa al otro.
    - Falla: si un proceso es mucho más lento, el rápido queda esperando **aunque el lento esté fuera de su región crítica**. Viola la **condición 3**.
    - ⚠ **Dije que violaba la condición 2; viola la 3.** El que bloquea no estaba usando la región crítica.
- ✓ **Solución 4 — Peterson** (1981; reemplazó a la solución más compleja de **Dekker**)
    - Código: `otro = 1 - proceso; interesado[proceso] = TRUE; turno = proceso; while (turno == proceso && interesado[otro] == TRUE) /* nada */;` y al salir `interesado[proceso] = FALSE;`.
    - **Dos variables**: `interesado[]` (quién quiere entrar) y `turno` (quién escribió último).
    - 🔔 **El último que escribe `turno` cede.** Como `turno` es **una sola variable**, no pueden estar esperando los dos.
    - 🔔 **Tabla de verdad de la condición de espera `(turno == yo Y interesado[otro])`**: de las 4 combinaciones, **solo una hace esperar** (las dos verdaderas).
    - 📌 **Lo que libera al que gira no es un cambio de `turno`**: es que el otro ponga `interesado[otro] = FALSE` al salir.
    - Sin competencia (el otro no está interesado), entra directo.
    - 🔑 **Peterson se inventó para sincronizar completamente por software**; TSL necesita un bus que se pueda bloquear (→ U2: 10.3 — Sincronización de multiprocesadores).

- ✓ **Solución 5 — La instrucción TSL** (hardware)
  - **`TSL REGISTRO, CANDADO`** (Test and Set Lock) — lee el candado y lo copia al registro, y guarda un valor distinto de cero en el candado, **en una sola operación indivisible**.
  - Código: `entrar_region: TSL REGISTRO,CANDADO / CMP REGISTRO,#0 / JNE entrar_region / RET` y `salir_region: MOVE CANDADO,#0 / RET`.
  - 🔔 **Cómo se lee**: una etiqueta con `:` no es una instrucción; `CMP` compara y anota el resultado; `JNE` lee ese resultado y salta si no era igual. **`CMP` + `JNE` juntos son un `if`.**
  - 🔔 **Las dos rarezas**: TSL **escribe 1 siempre** (inofensivo si ya era 1), y lo que te dice si ganaste es el **valor VIEJO** que quedó en el registro. **El registro es "tu mano": privado de cada proceso.**
  - 🔔 Mi reformulación, correcta: *"es lo mismo que la variable candado, solo que esa se hacía en dos pasos y había un hueco; TSL usa el hardware para hacer los dos pasos en uno"*.
  - 📌 **TSL bloquea el BUS DE MEMORIA**, no "a las otras CPU": ninguna CPU puede acceder a la memoria hasta que termine. Deshabilitar interrupciones bloquea **una sola CPU**.
  - **XCHG** — alternativa de los Intel x86: intercambia atómicamente un registro con una posición de memoria.
  - **? "El TSL siempre pone el mutex en 1, sin importar si ya estaba en 1"** — correcto; y si ya era 1, no cambia nada.
- **Peterson y TSL son correctos, pero las dos usan espera ocupada**: gastan CPU y producen el problema de fondo ↓.
- **Inversión de prioridades**
  - **Escenario**: H (alta prioridad) y L (baja); el planificador ejecuta a H siempre que esté listo. L está en su región crítica y H pasa a LISTO (por ejemplo, termina su E/S). El planificador pasa a H a ejecución, y H empieza a girar esperando el candado. **L nunca vuelve a recibir la CPU, así que nunca sale de su región crítica: H gira para siempre.**
  - 🔔 **Los 3 ingredientes**: prioridades estrictas + espera ocupada + el de baja prioridad tiene el recurso que necesita el de alta.
  - 🔔 **La oficina con UNA SOLA silla** (la CPU) y una llave (el candado): *"para hacer cualquier cosa —incluso devolver la llave— hay que estar sentado"*.
  - 🔔 **La frase que lo cierra**: *"para liberar el candado, L tiene que EJECUTAR `candado = 0`, y para ejecutar cualquier instrucción necesita la CPU"*.
  - 🔔 **Separar dos columnas: quién tiene la CPU** / **quién tiene el candado.** La fila clave: **L está en LISTO pero con el candado agarrado.**
  - ⚠ **Creía que "el de mayor prioridad nunca llega a ejecutarse".** Es al revés: **H consigue la CPU siempre; lo que no consigue es el candado.** La prioridad no falla: se respeta de más.
  - ⚠ **Creía que al acabarse el quantum H "pasa a bloqueado".** No: fin de quantum → **LISTO**, y como H es el más prioritario el planificador **lo vuelve a elegir**. Con espera ocupada **ninguna de las 4 instrucciones de TSL es una llamada al sistema**, así que **H no tiene con qué bloquearse**.
  - ⚠ **Tendí a culpar a TSL.** TSL garantiza exclusión mutua perfecta: **el culpable es la espera ocupada**, que le pasa igual a Peterson, a XCHG y a la variable candado.
  - 📌 **"Girar 1000 veces y después dormir un rato"** no lo resuelve del todo: no hay cómo elegir cuánto dormir sin suponer velocidades; se pierde la garantía de prioridad; se gastan igual 1000 vueltas; y con un tercer proceso M de prioridad media se vuelve a colgar (**dormir garantiza que H suelte la CPU, no que L la reciba**). 🔔 **Dormir por tiempo ≠ dormir por evento**; lo segundo es el semáforo (→ U2: Tema 7.2 — Semáforos).
  - **Herencia de prioridades** (complemento elaborado, no está en el libro) — el que tiene el candado hereda la prioridad del que lo espera. Caso real: el **Mars Pathfinder** (1997).
  - 🔗 **La misma anomalía vuelve en los multiprocesadores** (el spin lock al vencer el quantum) y se resuelve con la planificación inteligente (→ U2: 10.4 — Planificación de multiprocesadores).
- **Apropiativa contra no apropiativa** (adelanto del tema 8) — **no apropiativa**: el proceso suelta la CPU solo cuando se bloquea o termina; **apropiativa**: el sistema se la puede sacar (por ejemplo, con la interrupción de reloj). 🔗 (→ U2: 8.1 — Conceptos y métricas).

- 🔑 **Regla general que sale de acá**: *nunca te quedes inactivo —ni girando ni dormido— con un recurso tomado que otro necesita para desbloquearte* (→ U2: La regla precisa sobre dormirse con un candado tomado).

Tema 7.1 — Dormir y despertar

📖 cap. 2.3.4, págs. 125–128 (fig. 2-27)

🗒 Repaso U2, pág. 60 — "Tema 7.1 — Dormir y despertar (sleep y wakeup)"

- **`sleep()` y `wakeup(proceso)`** — **llamadas al sistema**: `sleep` bloquea a quien la llama hasta que otro lo despierte; `wakeup` despierta a un proceso. A diferencia de la espera ocupada, el que espera **no gasta CPU**.
- **Productor-consumidor (búfer limitado)** — un búfer de N ranuras compartido; el productor pone elementos y el consumidor los saca. La variable compartida `cuenta` lleva cuántos hay. Si el búfer está lleno, el productor duerme; si está vacío, duerme el consumidor.
  - Código: el productor hace `if (cuenta == N) sleep(); insertar; cuenta++; if (cuenta == 1) wakeup(consumidor);` y el consumidor `if (cuenta == 0) sleep(); quitar; cuenta--; if (cuenta == N-1) wakeup(productor);`.
  - 🔔 `if (cuenta == 1) wakeup(consumidor)` se lee: *"si después de meter el mío hay 1, antes había 0, así que el consumidor debe estar dormido"*.
  - 🔔 **La panadería**: el panadero (productor) pone panes en la vitrina de 100 bandejas (el búfer); el vendedor (consumidor) los saca.
- 🔢 **La señal de despertar perdida**
  - `cuenta` = 0 (búfer vacío).
  - El consumidor lee `cuenta` = 0 y decide dormirse… pero **lo interrumpen antes de dormirse**. Está en LISTO.
  - El productor inserta un elemento, `cuenta` = 1, y como vale 1 hace `wakeup(consumidor)`.
  - **El consumidor no está dormido**: la señal se pierde (no se guarda en ningún lado).
  - El consumidor vuelve, tiene guardado el 0 que leyó, y **se duerme**.
  - El productor llena el búfer y también se duerme. **Los dos duermen para siempre.**
  - 🔔 **Las dos causas, separadas**: `cuenta` no está protegida (leer y dormirse son dos pasos, otra vez el hueco) **y** `wakeup` no tiene memoria.
  - **El parche: el bit de espera de despertar** (la "alcancía") — guarda un wakeup enviado a alguien que no estaba dormido. No escala: **un bit recuerda "pasó algo", no "pasaron 5 cosas"**. De ahí sale el semáforo.
  - **? "Hilos" o "procesos" da igual acá**: el problema es el mismo para los dos.

Tema 7.2 — Semáforos

📖 cap. 2.3.5, págs. 128–130 (fig. 2-28)

🗒 Repaso U2, pág. 64 — "Tema 7.2 — Semáforos"

- **Semáforo** (Dijkstra, 1965) — una variable entera que **cuenta los despertares guardados**, con una cola de procesos bloqueados. Es la "alcancía con contador".
  - 🔔 **La lata de fichas**: `down` pide una ficha (si no hay, te dormís en la cola); `up` deja una ficha (y si hay alguien en la cola, lo despierta para que **complete su `down`**).
  - **`down(s)`** — si s > 0, la decrementa y sigue; si s = 0, el proceso se **bloquea** sin completar el down.
  - **`up(s)`** — incrementa s; si había procesos dormidos en ese semáforo, el sistema elige uno y lo despierta. **`up` nunca bloquea.**
  - 🔔 **`down` = pido una unidad del recurso y PUEDE bloquear; `up` = entrego una unidad y NUNCA bloquea.**

    - ⚠ **Pensaba `up` y `down` como "sumar" y "restar"**, en vez de "entregar" y "pedir". **No hay forma de esperar con un `up`**, así que un `up` nunca puede servir para pedir lugar. **`vacias` y `llenas` cuentan recursos, no eventos.**
    - 📣 **Un `up` sin destinatario SE GUARDA; un `wakeup` sin destinatario SE PIERDE.**
    - ⚠ Vocabulario: dije *"las llamadas perdidas acumuladas"*. Si se acumulan **no están perdidas**: están **guardadas**.
- **Atomicidad de `down` y `up`** — son **llamadas al sistema** y el SO **deshabilita las interrupciones** un instante mientras prueba y actualiza el semáforo; en un multiprocesador además protege el semáforo con **TSL o XCHG**.
    - 📌 **Eso no es espera ocupada**: la operación del semáforo dura microsegundos; la región crítica dura lo que dure.
    - **?** ⚠ **"Los semáforos usan TSL para bloquear el bus y que no haya interrupciones"** mezcla dos cosas: **deshabilitar interrupciones** protege en **una** CPU; **TSL** bloquea el **bus** para las **otras** CPU. Se usan las dos, cada una para lo suyo.
- **Productor-consumidor con tres semáforos**
    - **`mutex = 1`** — exclusión mutua sobre el búfer (uso: **exclusión mutua**).
    - **`vacias = N`** — ranuras vacías (uso: **sincronización**).
    - **`llenas = 0`** — ranuras llenas (uso: **sincronización**).
    - **Productor**: producir; `down(&vacias)`; `down(&mutex)`; insertar; `up(&mutex)`; `up(&llenas)`.
    - **Consumidor**: `down(&llenas)`; `down(&mutex)`; quitar; `up(&mutex)`; `up(&vacias)`; consumir.
    - 📣 **Invariante: `vacias + llenas = N`.** Cada uno consume de un semáforo y repone en el otro.
    - **Semáforo binario** — inicializado en 1 y usado por dos o más procesos para que uno solo entre a la región crítica.
    - 📌 **El orden de los `down` NO es decorativo**: si el productor hace `down(&mutex)` antes que `down(&vacias)` con el búfer lleno, **se duerme con el mutex puesto** y el consumidor nunca puede entrar a sacar: **interbloqueo**.
    - 📣 **Regla: primero el `down` de sincronización, después el de exclusión mutua.**
    - 🔗 Es el primer caso de la regla del candado (→ U2: La regla precisa sobre dormirse con un candado tomado).
    - ⚠ El interbloqueo lo expliqué bien, pero dije que los dos quedaban "ocupando tiempo de CPU". **Bloqueado = fuera de la cola de listos: no consume ni un ciclo.**
    - ⚖ **Espera ocupada contra interbloqueo**: espera ocupada = no avanza **y gasta CPU**; interbloqueo = no avanza **y NO gasta CPU**.
    - **? Deadlock e interbloqueo son lo mismo** (la traducción). Se desarrolla en la U6.

Tema 7.3 — Mutexes

📖 cap. 2.3.6, págs. 130–134 (figs. 2-29 y 2-30)

🗐 Repaso U2, pág. 69 — "Tema 7.3 — Mutexes"

- **Mutex** — versión simplificada del semáforo, **sin contador**: solo dos estados. Sirve para exclusión mutua.
    - 📣 **El mutex es un semáforo al que le sacaron el contador** (no al revés: el semáforo no es "mutex + contador").
    - ⚖ **Los números, al revés**: en el **mutex**, **0 = abierto** y distinto de 0 = cerrado; en el **semáforo**, 1 = **hay una ficha disponible**. (Mi observación del 29/10, correcta.)
    - **`mutex_lock`** — si está abierto, lo cierra y sigue; si está cerrado, el hilo espera. **`mutex_unlock`** — lo abre y, si había hilos esperando, uno puede entrar.
- **Implementación** (fig. 2-29) — `mutex_lock: TSL REGISTRO,MUTEX / CMP REGISTRO,#0 / JZE ok / CALL thread_yield / JMP mutex_lock / ok: RET` y `mutex_unlock: MOVE MUTEX,#0 / RET`.
    - 📣 **Es igual a `enter_region` hasta la tercera instrucción**: cambia el `JNE` (girar) por **`CALL`**

`thread_yield` (ceder). Con ese solo cambio **se cae la inversión de prioridades**.
    - **? ¿Quién ejecuta `thread_yield`? El hilo que quiere entrar y encuentra el mutex cerrado**: cede la CPU para que otro (quizás el que tiene el mutex) avance.
    - 💡 **`thread_yield` no es una llamada al sistema**: es el planificador de hilos en espacio de usuario, sin TRAP ni modo kernel. Por eso el caso sin conflicto son 3 instrucciones y los mutexes de usuario son **baratos**.
    - **? ¿El bucle `JMP mutex_lock` es espera ocupada?** No: en cada vuelta **cede la CPU** (queda LISTO), en vez de quemar el quantum.
- **Los tres niveles de espera**
    - `enter_region` con TSL → **gira sin ceder** → el hilo queda **EN EJECUCIÓN** quemando quantum.
    - `mutex_lock` de usuario → **gira cediendo** → el hilo queda **LISTO** y vuelve a intentar.
    - Mutex con bloqueo real → **no gira, se encola** → el hilo queda **BLOQUEADO**.
    - 💡 Son tres estados distintos del diagrama del tema 2. **Toda la unidad es la misma pregunta: cuando no podés avanzar, ¿en qué estado te quedás?** (→ U2: Tema 2 — Estados del proceso y el BCP)
- **`mutex_trylock`** — intenta tomar el mutex y, si no puede, **devuelve un código de falla en vez de bloquear**.
- **Pthreads** — `pthread_mutex_init`, `_destroy`, `_lock`, `_trylock`, `_unlock`.
    - **Variables de condición** — `pthread_cond_wait` (bloquea hasta una señal), `pthread_cond_signal` (despierta a uno), `pthread_cond_broadcast` (despierta a todos).
    - 📌 **Las variables de condición NO tienen memoria**: una señal sin nadie esperando **se pierde** (como el `wakeup`). Por eso van siempre con un mutex.
    - 💡 **`cond_wait` suelta el mutex atómicamente al dormirse**: si no, quedaría un hueco entre soltar y dormirse en el que se perdería la señal.
- 💡 **Atomicidad no es "terminar antes de que llegue la interrupción": es que nadie pueda observar el estado intermedio.** Dos formas: **TSL/XCHG** (bloquea el bus: leer + escribir el candado) y **llamada al sistema** (el kernel deshabilita interrupciones: soltar + dormirse, `down`, `up`).
- **Agujeros del "no hay reloj entre hilos de usuario"** — en un **multiprocesador** el hilo hermano corre en paralelo, de verdad; y **Pthreads en Linux es 1:1** (hilos de kernel, con quantum propio), así que en la práctica te pueden desalojar en cualquier instrucción.
- **Mutex contra semáforo binario** — el mutex solo hace exclusión mutua; el semáforo además puede **contar** (un mutex no puede valer "quedan 3 ranuras").
    - **Complemento elaborado (no está en el libro)**: el mutex real tiene **dueño** (solo lo abre quien lo cerró) y eso habilita la herencia de prioridades. Linux implementa los mutexes con **futex** (gira un poco en usuario y baja al kernel solo si hay conflicto).

Tema 7.4a — Monitores

📖 cap. 2.3.7, págs. 134–140 (figs. 2-33, 2-34 y 2-35)

🗐 Repaso U2, pág. 76 — "Tema 7.4a — Monitores"

- **Monitor** (Hoare 1974, Brinch Hansen 1975) — una colección de procedimientos, variables y estructuras de datos agrupados en un módulo. Los procesos pueden llamar a sus procedimientos, pero no tocar sus datos internos desde afuera.
    - 📌 **Es una construcción DEL LENGUAJE**: lo garantiza **el compilador**. *"Sólo puede haber un proceso activo en un monitor en cualquier instante."* (C no tiene monitores; el libro usa Pidgin Pascal.)
    - 💡 **Niveles**: TSL → hardware; `down`/`up` → SO; monitor → compilador.
- **Variables de condición del monitor** — `wait(c)` bloquea al proceso **y lo saca del monitor** (deja entrar a otro); `signal(c)` despierta a uno que estaba esperando en `c`.

- **¿Qué pasa después del `signal`?**
  - **Hoare** — corre el **despertado** y el que señaló se suspende.
  - **Brinch Hansen** — el que hace `signal` **sale del monitor de inmediato**: `signal` tiene que ser la **última instrucción** del procedimiento.
  - **Tercera solución** — el que señala sigue, y el despertado compite cuando el otro sale.
  - 📌 **El libro adopta Brinch Hansen**, *"debido a que, en concepto, es más simple y también más fácil de implementar"*: con esa regla, **`signal` tiene que ser la última instrucción** del procedimiento de monitor.
  - Si hay varios esperando en la misma variable, `signal` revive a **uno solo**, el *"que determine el planificador del sistema"*.
  - Complemento elaborado: con **varios consumidores**, el `if cuenta = 1` de la fig. 2-34 se queda corto (pueden quedar dos dormidos y se señaliza a uno); ahí sería más correcto un `signal` incondicional.
- 📌 **Las variables de condición no son contadores**: la señal sin oyente **se pierde**, así que `wait` tiene que ir **antes** que `signal`.
- 🌛 **¿No es sleep/wakeup otra vez? No**: la exclusión mutua automática garantiza que el `wait` se complete antes de que el otro pueda hacer `signal` — *"al consumidor ni siquiera se le va a permitir entrar al monitor"*. **Tercera forma de atomicidad**: hardware / kernel / **el compilador impide que otro entre**.
- **Productor-consumidor con monitor** (fig. 2-34) — `insertar` hace `if cuenta = N then wait(lleno)` … `if cuenta = 1 then signal(vacio)`; `eliminar` hace `if cuenta = 0 then wait(vacio)` … `if cuenta = N-1 then signal(lleno)`.
  - 🔑 **La asimetría central del tema 7**: el `signal` del monitor **se pierde** si no hay oyente → conviene condicionarlo (`if cuenta = 1`); el `up` del semáforo **se guarda** → va **siempre, sin condición** (condicionarlo rompe `vacias + llenas = N`).
  - ⚠ **Invertí esa asimetría**: dije que con semáforos también habría que condicionar el `up`. Es al revés. Después la reformulé bien: *"el monitor se fija si hay alguien dormido; en el semáforo depende de los números"*.
  - ⚠ **Creí que una señal perdida es siempre un error.** No: **señal perdida con alguien esperándola = desastre** (el wakeup perdido); **sin nadie esperando = trabajo al vacío**. El `if cuenta = 1` es una **optimización**, no una condición de correctitud.
  - 🌛 El `signal` le habla a **personas** (si no hay nadie dormido, no le habla a nadie); el `up` cuenta **cosas** (la ficha queda guardada).
- **Java** — `synchronized` en los métodos (exclusión mutua) y `wait`/`notify` como variables de condición.
- **Lo que el monitor no resuelve** — necesita un lenguaje que lo tenga, y como todo lo anterior **requiere memoria compartida**: no sirve entre máquinas.
- **?** Mi reformulación del monitor ("ejecuta de manera automática y no deja entrar a otro") es correcta: la exclusión mutua la pone el compilador, no el programador.

Tema 7.4b — Paso de mensajes y barreras

📖 cap. 2.3.8 y 2.3.9, págs. 140–145 (figs. 2-36 y 2-37)

🗐 Repaso U2, pág. 85 — "Tema 7.4b — Paso de mensajes y barreras"

- **`send(destino, &mensaje)` y `receive(origen, &mensaje)` — llamadas al sistema** (como los semáforos; no son construcciones del lenguaje como los monitores). `receive` puede bloquear hasta que llegue un mensaje.
  - 📌 **No requieren memoria compartida**: es el único mecanismo que sirve entre máquinas distintas.
  - ⚠ **Creí que exige procesos separados** y que no se puede usar entre hilos ni en una sola máquina. Al revés: **no NECESITA memoria compartida, pero funciona igual si la hay**; es el mecanismo **más general**. Si no se usa con memoria compartida es por el **costo de copiar**.

- **Problemas de diseño que los otros no tenían**
  - Mensajes perdidos → **acuse de recibo**.
  - Acuse perdido → mensaje duplicado → **números de secuencia**.
  - **Nomenclatura** de procesos y **autenticación** (¿con quién hablo de verdad?).
  - **Rendimiento**: *"copiar mensajes es siempre más lento que una operación con un semáforo o entrar a un monitor"*.
- **Productor-consumidor con N mensajes** (fig. 2-36) — el consumidor arranca mandando **N mensajes vacíos**; el productor toma un vacío, lo llena y lo devuelve.
  - 🔔 **El total de mensajes es constante = N**: el vacío es la ficha de `vacias` y el lleno la de `llenas`. Hacen **control de flujo** y dan **memoria acotada** (si no, el productor podría mandar 10.000 mensajes y el SO tendría que guardarlos todos).
  - Mi imagen de la "caja de transporte" era buena; me faltó la función (control de flujo).
- **Direccionamiento** — a **procesos** o a **buzones** (un lugar con capacidad para cierta cantidad de mensajes). El extremo sin búfer es el **rendezvous** (el `send` espera al `receive`). **MPI** es el estándar en cómputo paralelo.
- **Barreras** — para **grupos** de procesos que avanzan por fases: el que llega a la barrera se bloquea hasta que **llega el último**.
  - 🚀 **El grupo avanza al ritmo del más lento**: la misma lógica del `make` paralelo y de las pandillas (→ U2: 10.4 — Planificación de multiprocesadores).
- **El cuadro de las primitivas del tema 7** (las tres preguntas que lo resumen)
  - **¿Gira o se bloquea?** — giran: variable candado, alternancia estricta, Peterson, TSL/XCHG (deshabilitar interrupciones no gira, pero frena a todos los demás). Se bloquean: sleep/wakeup, semáforos, mutex (cede o se bloquea), monitores, mensajes, barreras.
  - **¿La señal se guarda o se pierde?** — se guarda: semáforo (`up`), mensajes (en el buzón). Se pierde: `wakeup`, `signal` del monitor, variable de condición de Pthreads.
  - **¿Necesita memoria compartida?** — todos sí, **menos el paso de mensajes**.
- ⚖ **Memoria compartida contra paso de mensajes** — compartir es rápido pero peligroso; aislar es seguro pero caro. 🚀 (→ U2: Tema 3 — Hilos: el modelo clásico), (→ U1: Tema 9 — Estructuras de los sistemas operativos) y los multiprocesadores basados en directorios, que internamente pasan mensajes (→ U2: 10.1 — Hardware de multiprocesador).

Tema 8 — Planificación de la CPU ★

📖 cap. 2.4, págs. 145–163 (figs. 2-38 a 2-43) + apunte de la cátedra (le dice **SJN** al SJF y **SRTN** al SRTF)

🗐 Repaso U2, pág. 91 — "Tema 8.1 — Planificación: conceptos y métricas"

El tema que más puntos vale en los prácticos. Mapa del tema: 8.1 conceptos y métricas → 8.2 algoritmos por lotes → 8.3 interactivos → 8.4 tiempo real → 8.5 política contra mecanismo → 8.6 planificación de hilos.

8.1 — Conceptos y métricas

📖 cap. 2.4.1, págs. 145–152 (figs. 2-38 y 2-39)

🗐 Repaso U2, pág. 91 — "Tema 8.1 — Planificación: conceptos y métricas"

- **Planificador** — la parte del SO que elige qué proceso se ejecuta; usa un **algoritmo de planificación**. **Solo elige entre los LISTOS** (→ U2: Tema 2 — Estados del proceso y el BCP).
  - Importa poco en una PC (casi siempre hay un solo proceso activo) y mucho en un servidor o una mainframe.
- **Conmutar no es gratis** — pasar a modo kernel, guardar el estado en el BCP, cambiar el mapa de memoria e **invalidar la caché** (→ U2: Manejo de una interrupción y cambio de contexto). Por eso no se puede conmutar todo el tiempo.

- **Ráfagas** — un proceso alterna **ráfagas de CPU** (calcula) con **esperas de E/S**.
  - **? Ráfaga** = el tramo de tiempo que el proceso usa la CPU sin parar, hasta que pide E/S (o hasta que lo sacan).
  - **Limitado a cálculos (CPU-bound)** — ráfagas de CPU **largas** y esperas de E/S poco frecuentes. Ej.: comprimir un video, multiplicar matrices.
  - **Limitado a E/S (I/O-bound)** — ráfagas de CPU **cortas** y esperas de E/S frecuentes. Ej.: un editor que espera teclas.
  - 📌 **El factor clave es la longitud de la ráfaga de CPU, NO la de la E/S.** *"Están limitados a la E/S debido a que no realizan muchos cálculos entre una petición de E/S y otra, no debido a que tengan peticiones de E/S en especial largas."*
  - ⚠ **Cuando clasifiqué A (2 ms de CPU + 50 ms de disco) usé la E/S para justificar.** La columna de E/S no se mira nunca: es el distractor.
  - Copiar a la RAM de video **cuenta como cálculo**, no como E/S.
  - Con CPUs cada vez más rápidas, los procesos tienden a ser más **limitados a E/S**.
- **Cuándo hay que planificar**
  - Al **crear** un proceso (¿corre el padre o el hijo?).
  - Al **terminar** un proceso.
  - Cuando un proceso se **bloquea** (E/S, semáforo, etc.).
  - Cuando llega una **interrupción de E/S** (alguien pasó a LISTO).
  - Y, en los sistemas apropiativos, en cada **interrupción de reloj**.
  - **?** Sí: cuando se acaba el quantum, la interrupción de reloj llama al planificador, que elige quién sigue.
- **Apropiativa contra no apropiativa**
  - **No apropiativa** — el proceso corre hasta que se **bloquea** o **libera la CPU** voluntariamente. Sin reloj es la única opción.
  - **Apropiativa** — el proceso corre a lo sumo un tiempo fijo; al terminar ese tiempo, el planificador lo **suspende** (→ LISTO) y elige otro. Necesita una **interrupción de reloj**.
- **Las tres categorías de entornos** — **lotes** (sin usuarios esperando; se aceptan no apropiativos o con períodos largos), **interactivos** (hace falta apropiación), **tiempo real** (a veces no hace falta apropiación, porque los procesos saben que no pueden correr mucho).
- **Metas** (fig. 2-39)
  - **Todos los sistemas** — **equidad** (a cada uno su parte justa), **aplicación de políticas**, **balance** (todas las partes ocupadas).
  - **Lotes** — **rendimiento** (trabajos por hora), **tiempo de retorno** (minimizarlo), **utilización de la CPU** (mantenerla ocupada).
  - **Interactivos** — **tiempo de respuesta** (responder rápido), **proporcionalidad** (cumplir las expectativas del usuario).
  - **Tiempo real** — **cumplir los plazos**, **predictibilidad**.
- **Las métricas** — lo que te hacen calcular.
  - **Tiempo de retorno = fin − llegada.** (La cátedra: "desde el envío hasta la terminación".)
  - **Tiempo de espera = retorno − ráfaga.** Sirve **siempre**, con o sin apropiación.
  - **Tiempo de respuesta = primer arranque − llegada.**
  - 📌 `inicio - llegada` como espera **solo sirve sin apropiación**.
  - 🌤 **Retorno = espera + ráfaga** (se suman, no compiten). **Respuesta ≤ espera ≤ retorno, siempre.**
  - 🌤 **La respuesta mide solo el primer tramo de espera; la espera mide todos los tramos.** Sin apropiación hay un solo tramo y coinciden; con RR o SRTF la respuesta es chica pero la espera crece.
  - ⚠ El compañero que decía "la espera no puede ser 10 si esperó 12 como máximo" confundía retorno con espera: P3 llegó en 2, no en 0.

- 🔢 **Ejemplo resuelto de FCFS** — P1 (llega 0, ráfaga 8), P2 (1, 4), P3 (2, 2)
  - Gantt: `P1[0–8] P2[8–12] P3[12–14]`.
  - Retornos: P1 = 8 – 0 = 8; P2 = 12 – 1 = 11; P3 = 14 – 2 = 12 → **promedio 31/3 = 10,33**.
  - Esperas (retorno – ráfaga): 0; 7; 10 → **promedio 17/3 = 5,67**.
  - Respuestas: 0; 7; 10 (en FCFS coinciden con las esperas).
  - 🔔 **P3 necesitaba 2 de CPU y tardó 12 en salir**: ése es el defecto de FCFS y el motivo de todos los demás algoritmos.

8.2 — Algoritmos por lotes: FCFS, SJF y SRTF

📖 cap. 2.4.2, págs. 152–154 (figs. 2-40 y 2-41)

🗐 Repaso U2, pág. 97 — "Tema 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF"

- **FCFS (primero en llegar, primero en ser atendido)** — **no apropiativo**, una sola cola de listos. Un proceso que se bloquea va **al final** de la cola cuando vuelve.
  - **? FCFS = First Come, First Served.**
  - Ventaja: fácil de entender y programar (una lista ligada).
  - Desventaja (ejemplo del libro): un proceso limitado a cálculos que corre 1 s cada vez y muchos limitados a E/S que hacen 1000 lecturas de disco cada uno → cada uno de esos tarda **1000 s** en vez de 10 s con apropiación cada 10 ms. **El largo frena a los cortos** (efecto convoy).
- **SJF (el trabajo más corto primero)** — **no apropiativo**; elige el de **ráfaga más corta**. La cátedra lo llama **SJN**.
  - 🔢 **Ejemplo del libro** (A=8, B=4, C=4, D=4, todos en t=0)
    1. En orden A, B, C, D: retornos 8, 12, 16, 20 → promedio **14**.
    2. SJF (B, C, D, A): retornos 4, 8, 12, 20 → promedio **11**.
  - 🔔 **"El primero pesa 4 veces"**: con 4 trabajos de duraciones a, b, c, d en ese orden, el retorno promedio es **(4a + 3b + 2c + d)** / **4**. El primero lo esperan los cuatro (él incluido), por eso cuenta 4 veces; el último solo se cuenta a sí mismo. Para minimizar, el más corto va primero.
  - 📌 **SJF es óptimo SOLO cuando todos los trabajos están disponibles al mismo tiempo.** Contraejemplo del libro: A–E con 2, 4, 1, 1, 1 que llegan en 0, 0, 3, 3, 3. SJF hace A, B, C, D, E → promedio 4,6; hacer B, C, D, E, A da 4,4.
  - **Desventajas** — necesita conocer las ráfagas **de antemano** (un dato del futuro) y puede producir **inanición** del largo. 🩹 (→ U2: El proceso más corto a continuación (SPN)).
  - 🔢 **SJF con el juego de la clase** — P1 (0, 8), P2 (1, 4), P3 (2, 2): `P1[0–8] P3[8–10] P2[10–14]` → retorno promedio **9,67**, espera **5,00**. P3 bajó su retorno de 12 a 8, pero poco: **P1 llegó solo, ocupó los primeros 8 instantes y SJF no puede sacarlo.**
  - 🔢 **Ejemplo de la condición de SJF** — cuando no llegan juntos, SRTF le gana: `SJF: P1[0–5] P2[5–6]` → retorno 5,00 contra `SRTF: P1[0–1] P2[1–2] P1[2–6]` → **3,50**.
- **SRTF (menor tiempo restante a continuación)** — versión **apropiativa** de SJF: si llega uno cuyo tiempo total es menor que **lo que le queda** al actual, lo desaloja. La cátedra lo llama **SRTN**.
  - 🔢 **SRTF con el juego de la clase** (verificado con el simulador)
    - t = 0: solo P1 (le faltan 8) → corre P1.
    - t = 1: llega P2 con 4; a P1 le faltan 7 → **P2 desaloja a P1**.
    - t = 2: llega P3 con 2; a P2 le faltan 3 → **P3 desaloja a P2**.
    - t = 4: P3 termina; quedan P2 (3) y P1 (7) → corre P2.
    - t = 7: P2 termina → corre P1 hasta 14.
    - Gantt `P1[0–1] P2[1–2] P3[2–4] P2[4–7] P1[7–14]` → retorno promedio **7,33**, espera **2,67**, respuesta **0**.
    - **Comparación del mismo juego**: FCFS 10,33 / 5,67 · SJF 9,67 / 5,00 · SRTF 7,33 / 2,67 · RR q = 2 9,00 / 4,33 (retorno / espera).

    - En SRTF un proceso puede tener **respuesta 0 y espera mayor**: lo desalojan después de arrancar.
- **Desempate de la cátedra** — *entre un proceso que llega y otro que sale de ejecución, entra primero el que llega* a la cola.
- 🌤️ **Ráfagas iguales y llegada simultánea** (A(0,3) y B(0,3)) → **FCFS, SJF, SRTF y RR con q = 3 dan lo mismo**: `A[0–3] B[3–6]`, respuesta de B = 3. Si un ejercicio da todas las ráfagas iguales, probablemente la respuesta es "dan lo mismo".
    - A en t = 3 **termina**, no se desaloja: no hay fin de quantum, hay fin de proceso.

8.3 — Round Robin

📖 cap. 2.4.3, págs. 154–155 (fig. 2-42)

🗒️ Repaso U2, pág. 100 — "Tema 8.3 — Round Robin"

- **Round Robin (turno circular)** — a cada proceso se le asigna un **quantum**; si al terminarlo sigue ejecutándose, se lo desaloja y va **al final** de la cola de listos; si se bloquea o termina antes, se conmuta en ese momento.
    - 🚀 Es el heredero del **tiempo compartido** de la tercera generación (→ U1: Tema 7 — Historia por generaciones).
- 🔢 **Cómo resolver un RR en el examen**
    1. Ordenar por llegada y poner en la cola a los que llegan en t = 0.
    2. Ejecutar al primero de la cola durante min(quantum, lo que le queda).
    3. En el instante en que termina ese tramo, **primero** encolar a los que llegaron hasta ese instante y **después** al que sale (si no terminó) — desempate de la cátedra.
    4. Repetir hasta vaciar la cola; anotar fin, retorno, espera y respuesta de cada uno.
    5. Si te piden "la cola en el instante t", escribila **después** de aplicar el paso 3 en ese instante.
- 🔢 **Ejemplo con P1(0,8) P2(1,4) P3(2,2) y q = 2**
    - `P1[0–2] P2[2–4] P3[4–6] P1[6–8] P2[8–10] P1[10–14]`.
    - Retorno promedio **9,00**; espera promedio **4,33**; respuesta promedio **1,00**.
    - Con q = 1: retorno 9,33 y respuesta 0,33. Con q = 3: retorno 10,33 y respuesta 2,00. Con q = 4: retorno 9,67 y respuesta 3,00. **Con q = 8 queda `P1[0–8] P2[8–12] P3[12–14]`: idéntico a FCFS** (retorno 10,33, espera 5,67).
- **El problema del quantum**
    - **Muy corto** → demasiadas conmutaciones y se **desperdicia CPU**. Ej.: quantum de 4 ms con conmutación de 1 ms → 1 / (4 + 1) = **20 %** de CPU perdida.
    - **Muy largo** → mala **respuesta** a las peticiones interactivas. Con quantum enorme, RR **degenera en FCFS**.
    - 🌤️ Un quantum de **20 a 50 ms** suele ser un compromiso razonable.
    - 🌤️ **(n – 1) · q** (complemento): con n procesos en la cola, el último espera como mucho (n – 1) quantums antes de tocar la CPU (sin contar las conmutaciones).
    - Mi respuesta, correcta: con 500 ms empeora el **tiempo de respuesta** (tirones); con 1 ms se **desperdicia CPU** en conmutaciones.

8.3b — Planificación por prioridades y múltiples colas

📖 cap. 2.4.3, págs. 155–158

🗒️ Repaso U2, pág. 104 — "Tema 8.3b — Planificación por prioridades"

- **Por prioridad** — a cada proceso se le asigna una prioridad y corre el **listo de mayor prioridad**.
    - **Estáticas** (las fija alguien de afuera: decanos, profesores, alumnos; o `nice` en UNIX) contra **dinámicas** (las ajusta el sistema).
    - **No apropiativa** contra **apropiativa** (el que llega con más prioridad desaloja al actual).

    - 🔢 **Ejemplo verificado**: L (llega 0, ráfaga 6, prioridad 3), M (2, 4, p2), H (4, 3, p1), con 1 = más prioritario.
        1. No apropiativa: `L[0–6] H[6–9] M[9–13]` → retorno 7,33, espera 3,00, respuesta 3,00.
        2. Apropiativa: `L[0–2] M[2–4] H[4–7] M[7–9] L[9–13]` → retorno 7,67, espera 3,33, respuesta 0.

- **El problema: inanición** — los de baja prioridad pueden no correr nunca.
    - **Soluciones** — bajar la prioridad del proceso en ejecución en cada pulso de reloj; darle un **quantum máximo**; subir la prioridad de los que esperan mucho (**envejecimiento de prioridad**).
    - 🔗 **Inanición + candado = inversión de prioridades** (→ U2: Inversión de prioridades). Si el de prioridad 9 tiene el candado X y el de prioridad 1 lo pide, el más importante queda frenado por el menos importante. **En el parcial, nombrá las dos**: "inversión de prioridades causada por la inanición del proceso de prioridad baja".

- **Prioridad dinámica** `1/f` — f = fracción del último quantum que usó el proceso. Favorece a los limitados a E/S.
    - Ejemplo: usó 1 ms de un quantum de 50 → f = 1/50 → prioridad **50**; usó 25 ms → **2**; usó todo → **1**.
    - Ejemplo de la clase: un interactivo que usa 2 ms de 50 (f = 0,04) → **25**; uno de cálculo puro (f = 1) → **1**.
    - **? "Cálculo puro"** = un proceso que solo calcula y nunca pide E/S: usa el quantum completo y siempre lo tienen que echar.
    - ⚠ **Convención**: con `1/f`, **el número más grande es el más prioritario**, al revés de la cátedra (**1 = más prioritario**). **Declarar siempre la convención en la respuesta.** Lo que no cambia nunca: *poca CPU → favorecer; mucha CPU → castigar.*
    - ⚠ El ejemplo de prioridades dinámicas del otro chat usa "número mayor = más prioridad" y una regla de ±1 por tic que **no está en el libro**.

- **Clases de prioridad** — se agrupan los procesos en clases; entre clases se usa prioridad y dentro de cada clase, Round Robin.
- **Múltiples colas** (CTSS) — conmutar era caro (había que pasar el proceso a disco), así que se convino dar quantums más largos a los procesos de cálculo: la clase más alta corre **1 quantum**, la siguiente **2**, la siguiente **4**, luego 8…; el que agota su cuota baja una clase.
    - 🔢 Un proceso que necesita 100 quantums corre 1, 2, 4, 8, 16, 32 y 64 → **7 intercambios** en vez de 100.
    - Se abusó: al apretar Enter en la terminal el proceso subía a la clase más alta, y un usuario descubrió que apretar Enter al azar cada pocos segundos aceleraba sus trabajos.

8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativas

📖 cap. 2.4.3, págs. 158–160

🗐 Repaso U2, pág. 110 — "Tema 8.3c — Más corto a continuación, garantizada, lotería y partes equitativas"

- **El proceso más corto a continuación (SPN)** — SJF para interactivos: como no se conoce la próxima ráfaga, se **estima** con las anteriores.
    - 🔮 **Estimación = a · T_0 + (1 – a) · T_1** (T_0 = estimación anterior, T_1 = última ráfaga medida). A esta técnica se la llama **envejecimiento**.
    - Con a = ½, los pesos de las ráfagas pasadas son ½, ¼, ⅛, 1/16…: lo viejo se va olvidando.
    - 🔢 **Ejemplo verificado** — desde 10, con ráfagas 6, 4, 14 y 6 → estimaciones **8, 6, 10, 8**. Desde 20 con ráfagas de 8, 8, 8 → **14, 11, 9,5**, y luego 8,75, 8,375… (se acerca a 8 sin llegar). Una ráfaga de 40 en ese punto → ½ · 8,09 + ½ · 40 ≈ **24**.
    - **El parámetro a** controla cuánto se recuerda: a cerca de 1 olvida lento; a cerca de 0 olvida rápido.
    - ⚖ **Los dos "envejecimientos"**: en prioridades, envejecer = **subir la prioridad** de quien espera mucho (contra la inanición); en SPN, envejecimiento = **la fórmula de promedio** que estima la ráfaga. Mismo nombre, cosas distintas.

    - 🔗 El umbral dinámico de espera activa de Karlin (1991) es esta misma idea aplicada a candados (→ U2: 10.3 — Sincronización de multiprocesadores).

- **Planificación garantizada** — con n procesos, a cada uno le corresponde **1/n** de la CPU. Se calcula **razón = CPU consumida / CPU que le correspondía**, y corre el de **menor razón**.
    - 🔢 **Ejemplo verificado** — 5 procesos en 100 ms → 20 ms cada uno; consumieron A 10, B 30, C 20, D 25, E 15 → razones **0,50, 1,50, 1,00, 1,25, 0,75 → corre A**.
    - **? El 20 %** con 5 procesos sale de 1/n = 1/5.

- **Planificación por lotería ★** — se reparten **boletos**; en cada decisión se sortea uno y gana quien lo tiene. Un proceso con una fracción f de los boletos recibe **alrededor de f de la CPU**.
    - Más prioridad = más boletos. Responde de inmediato: un proceso nuevo con boletos ya participa en el próximo sorteo.
    - **Procesos cooperativos** pueden **intercambiar boletos** (el cliente le presta sus boletos al servidor mientras espera).
    - Resuelve problemas difíciles: un servidor de video con flujos de 10, 20 y 25 cuadros/s recibe 10, 20 y 25 boletos.
    - 🔢 **Ejemplo verificado** — A 5, B 15, C 30 boletos (total 50) → **10 %, 30 %, 60 %**. Si C le regala 20 a A: A 25, B 15, C 10 → **50 %, 30 %, 20 %** (el total sigue en 50).
    - ⚠ **Leí los boletos como porcentajes sin dividir por el total.** Siempre: porcentaje = boletos / total.
    - 📌 **¿Por qué no se puede hacer con prioridades?** Porque una prioridad es un **puesto (ordinal)**, no una **cantidad** divisible y transferible ("sos 3º", no "tenés 15"); y **la prioridad la fija el sistema**, no el proceso. Los boletos son tuyos y los regalás.
    - ⚠ La explicación del otro chat ("B con prioridad media 15 se metería en el medio") confunde boletos con números de prioridad.

- **Partes equitativas (fair-share)** — se reparte la CPU **por usuario**, no por proceso. Ej.: el usuario 1 tiene A, B, C y D, y el usuario 2 tiene E; con 50 % cada uno: A E B E C E D E…
- **Dos y tres niveles** — figuran en el programa pero **no están desarrollados en el Tanenbaum** de la cátedra (no hay que inventarlos). 🔗 En multiprocesadores sí aparece la *planificación de dos niveles* (→ U2: 10.4 — Planificación de multiprocesadores).
- 🏷 **Cuadro de los algoritmos** — FCFS: lotes, no apropiativo, convoy. SJF/SJN: lotes, no apropiativo, óptimo con llegada simultánea, necesita el futuro. SRTF/SRTN: lotes, apropiativo. RR: interactivo, apropiativo, el quantum es el problema. Prioridades: interactivo, puede ser las dos cosas, inanición. Múltiples colas: interactivo, quantums crecientes. SPN: interactivo, estima con envejecimiento. Garantizada: razón consumido/correspondiente. Lotería: boletos, proporcional y transferible. Partes equitativas: por usuario.

8.4 — Planificación de tiempo real

📖 cap. 2.4.4, págs. 160–161

🗒 Repaso U2, pág. 118 — "Tema 8.4 — Planificación de tiempo real"

- **Tiempo real duro** (plazos absolutos) contra **suave** (perder un plazo ocasional es tolerable) 🔗 (→ U1: Tiempo real duro contra suave).
- **Eventos periódicos** (ocurren a intervalos regulares) contra **aperiódicos** (impredecibles).
    - **? Evento** = algo externo al que el sistema tiene que responder (llega un dato del sensor, hay que producir un cuadro de video).

- **Condición de planificabilidad** — con m eventos periódicos, el evento i con período $\mathbf{P_i}$ y $\mathbf{C_i}$ segundos de CPU: $\mathbf{\Sigma\ C_i / P_i \leq 1}$.
    - 🔢 **Ejemplo del libro** — períodos 100, 200 y 500 ms con 50, 30 y 100 ms de CPU: 0,5 + 0,15 + 0,2 = **0,85 ≤ 1 → planificable** (15 % libre). Un cuarto evento de período 1 s admite hasta **150 ms** de CPU.

- 🔢 **Ejercicio de la clase** — A(50, 15), B(100, 20), C(200, 40) → 0,3 + 0,2 + 0,2 = **0,7**, libre 30 %; un evento D cada 500 ms admite hasta 0,3 × 500 = **150 ms**.
- 🌤️ **Cuanto más seguido ocurre un evento, menos puede trabajar cada vez.**
- 📌 **Trampa 1 — cuál va arriba**: es $\mathbf{C_i}$ / $\mathbf{P_i}$ (CPU que necesita / cada cuánto ocurre). Cada término tiene que dar menos que 1; si uno da más, lo invertiste (o ese evento solo ya es imposible).
- 📌 **Trampa 2 — unidades**: si los períodos están en segundos y la CPU en milisegundos, la cuenta sale 1000 veces mal. Pasar todo a la misma unidad antes de dividir.
- Variantes del ejemplo del libro que se hicieron en clase: sumas de **1,250**, **1,050** y **1,210** → las tres **no planificables**.
- 📌 **Los aperiódicos no están en la suma**: si llega una emergencia, come del margen libre.
- 📌 **Los $\mathbf{C_i}$ tienen que ser del PEOR caso**, no del promedio.
- 📌 **La condición es NECESARIA pero NO SUFICIENTE.** Ejemplo: A (cada 100 ms, 50 ms) y B (cada 10 ms, 2 ms) suman 0,70, pero con un planificador **no apropiativo** que arranca A, B termina en t = 52 y pierde su plazo. Hace falta capacidad **y** un algoritmo que la use bien (apropiativo, con prioridad al plazo más cercano).

- **Estáticos** (deciden antes de ejecutar; necesitan conocer todo de antemano) contra **dinámicos** (deciden en tiempo de ejecución). Los algoritmos concretos (RMS, EDF) el libro los da en el capítulo de multimedia, no acá.

8.5 — Política contra mecanismo

📖 cap. 2.4.5, págs. 161–162

🗐 Repaso U2, pág. 122 — "Tema 8.5 — Política contra mecanismo"

- **El problema** — el planificador no sabe cuál de los procesos de una aplicación es el más importante; el proceso padre sí lo sabe (por ejemplo, un sistema de base de datos con varios hijos).
- **La solución** — separar el **mecanismo** de planificación de la **política**: el kernel implementa un algoritmo parametrizado (por ejemplo, prioridades) y ofrece una **llamada al sistema** para que el proceso padre fije las prioridades de sus hijos. **El mecanismo queda en el kernel; la política la pone un proceso de usuario.**
- 🔗 **Es la misma regla del microkernel** (→ U1: Mecanismo en el kernel, política en el espacio de usuario) y vuelve en el servidor central de Tucker y Gupta en multiprocesadores (→ U2: 10.4 — Planificación de multiprocesadores).

8.6 — Planificación de hilos

📖 cap. 2.4.6, págs. 162–163 (fig. 2-43)

🗐 Repaso U2, pág. 123 — "Tema 8.6 — Planificación de hilos"

- **Hilos a nivel de usuario — el kernel elige PROCESO** — el kernel le da el quantum al proceso A, y el planificador de hilos de A elige A1. *"Como no hay interrupciones de reloj para multiprogramar hilos, este hilo puede continuar ejecutándose todo el tiempo que quiera"* (dentro del quantum de A).
  - Fig. 2-43, quantum 50 ms, ráfagas de 5 ms: secuencia posible **A1, A2, A3, A1, A2, A3**; **imposible A1, B1, A2, B2** (no aparece un B hasta que A agota su quantum).
- **Hilos a nivel de kernel — el kernel elige HILO** — puede intercalar hilos de procesos distintos: **A1, B1, A2, B2, A3, B3** es posible. El proceso "tiene un pequeño papel o ninguno".
- ☆ **El cuadro comparativo**
  - **Costo de conmutar** — usuario: unas pocas instrucciones; kernel: conmutación de contexto **completa** (mapa de memoria, caché) → órdenes de magnitud más lento.
  - **Si un hilo se bloquea por E/S** — usuario: se suspende **todo el proceso**; kernel: solo ese hilo.
  - **Planificador** — usuario: puede ser específico de la aplicación (en el servidor web, preferir al despachador); kernel: no conoce la aplicación, pero puede preferir un hilo **del mismo proceso** porque es más barato (afinidad). 🔗 (→ U2: 10.4 — Planificación de multiprocesadores).

- 🔔 **Regla de decisión: ¿qué hace esta aplicación MUCHAS veces? Si se bloquea mucho → hilos de kernel. Si conmuta mucho → hilos de usuario.**
    - ⚠ Justifiqué la elección de hilos de usuario con "puede ceder fácilmente"; lo que decide es **el precio de cada conmutación**.
    - 🔔 Dato que delata hilos de kernel: **se intercalan hilos de procesos distintos** (`A2, B1, A3`).

Tema 9 — Problemas clásicos de comunicación entre procesos

📖 cap. 2.5, págs. 163–168 (figs. 2-44 a 2-47)

🗐 Repaso U2, pág. 126 — "Tema 9.1 — Los filósofos comelones"

- **Barbero dormilón** — figura en el programa pero **NO está en el Tanenbaum 3ª ed.** Si hace falta, se da como complemento elaborado y marcado.

9.1 — Los filósofos comelones

📖 cap. 2.5.1, págs. 164–167 (figs. 2-44, 2-45 y 2-46)

🗐 Repaso U2, pág. 126 — "Tema 9.1 — Los filósofos comelones"

- **Planteo** (Dijkstra, 1965) — cinco filósofos en una mesa redonda, cinco platos y **un tenedor entre cada par de platos**. Para comer hacen falta **los dos tenedores** de los costados. Alternan entre **pensar** y **comer**.
    - Modela procesos que compiten por **acceso exclusivo a un número limitado de recursos**, como los dispositivos de E/S.
- **Intento 1 — tomar el izquierdo y después el derecho** (fig. 2-45) — si los cinco toman su izquierdo **a la vez**, ninguno consigue el derecho: **interbloqueo**.
- **Intento 2 — si el derecho no está, devolver el izquierdo, esperar y reintentar** — si todos arrancan juntos, toman, ven ocupado, sueltan y reintentan al mismo ritmo para siempre: **inanición** (todos los programas siguen ejecutándose, pero ninguno progresa).
- ⚖ **Interbloqueo contra inanición** (la comparación que se toma) — **interbloqueo**: todos **bloqueados** esperando algo que nunca va a llegar, **sin gastar CPU**; **inanición**: los procesos **siguen corriendo** (o esperando turno) pero alguno **nunca progresa**.
- **Intento 3 — esperar un tiempo aleatorio** — casi siempre funciona (como Ethernet), pero no sirve donde no se tolera ni una falla muy improbable (control de una planta nuclear).
- **Intento 4 — un mutex para toda la mesa** — es **correcto** (nunca hay interbloqueo), pero **come un solo filósofo por vez**, cuando con cinco tenedores pueden comer dos.
- **Variante del filósofo zurdo** — uno toma primero el derecho; rompe el ciclo de espera y **sí funciona** (no hay interbloqueo). Se lo conoce como **romper la simetría**: no agrega ni un semáforo, solo cambia el orden de dos líneas.
    - 🔑 Es la técnica de prevención más usada: **ordenar los recursos** para que no pueda haber **espera circular**, una de las cuatro condiciones del interbloqueo de la **U6**.
    - ⚠ **Contesté que no funcionaba**, porque confundí cuál es el tenedor derecho de F4: es el **tenedor 0** = (4 + 1) % 5, no el 4. F0 y F4 pelean por el 0, el tenedor 4 queda libre y F3 puede comer. Sí funciona: verificado con búsqueda exhaustiva de todos los estados.
- **La solución buena (fig. 2-46)**
    - **Las tres piezas** — el arreglo **`estado[N]`** (PENSANDO, HAMBRIENTO o COMIENDO), un **`mutex`** = 1 que protege `estado[]`, y un **semáforo por filósofo `s[i]`** = 0, donde se bloquea el que no puede comer.
    - **Vecinos** — `IZQUIERDO = (i + N − 1) % N` y `DERECHO = (i + 1) % N`.
        - ■ **?** El `% N` hace que la mesa sea redonda: el vecino derecho del filósofo 4 es el 0.
    - 🔢 **`tomar_tenedores(i)`**

1. `down(&mutex)` — entra a la región crítica.
2. `estado[i] = HAMBRIENTO`.
3. `probar(i)` — intenta tomar los dos tenedores.
4. `up(&mutex)` — sale de la región crítica.
5. `down(&s[i])` — se bloquea si no consiguió los tenedores.

- 🔢 **`poner_tenedores(i)`**
  1. `down(&mutex)`.
  2. `estado[i] = PENSANDO`.
  3. `probar(IZQUIERDO)` — ¿el vecino izquierdo puede comer ahora?
  4. `probar(DERECHO)` — ¿y el derecho?
  5. `up(&mutex)`.
- **`probar(i)`** — `if (estado[i] == HAMBRIENTO && estado[IZQUIERDO] != COMIENDO && estado[DERECHO] != COMIENDO) { estado[i] = COMIENDO; up(&s[i]); }`.
- 🔔 **El `down(&s[i])` va FUERA del mutex**: si el filósofo se durmiera con el mutex tomado, nadie podría entrar a `probar()` para despertarlo. 🔑 (→ U2: La regla precisa sobre dormirse con un candado tomado).
- 🔔 **El `up(&s[i])` de `probar(i)` se guarda** si el filósofo todavía no hizo su `down`: por eso no se pierde el aviso (→ U2: Un up sin destinatario SE GUARDA; un wakeup sin destinatario SE PIERDE.).
- 🔢 **Pares que pueden comer juntos**: F0+F2, F0+F3, F1+F3, F1+F4, F2+F4 (**5 pares**); **tríos: 0** → el máximo paralelismo es **2**.
- 🔑 **Mutex de mesa contra `estado[]`** es el mismo movimiento que hace el SMP: partir un candado grande en candados chicos (→ U2: 10.2 — Los tres tipos de SO multiprocesador).

- **Simulador**: `unidades/u02-procesos/visuales/filosofos_comelones.html` (los cuatro esquemas, con el botón para forzar el interbloqueo del libro y ver que nadie consume CPU).

9.2 — Lectores y escritores

📖 cap. 2.5.2, págs. 167–168 (fig. 2-47)

🗐 Repaso U2, pág. 136 — "Tema 9.2 — Lectores y escritores"

- **Planteo** (Courtois y colaboradores, 1971) — una base de datos (una reserva de vuelos). **Varios pueden leer a la vez**, pero si uno **escribe**, nadie más puede acceder, ni siquiera para leer.
  - ⚠ **Lo que lo diferencia de todo lo anterior**: no todos los accesos son iguales. Exclusión mutua solo **entre escritores** y **entre un escritor y los lectores**.
- **La solución (fig. 2-47)** — `mutex = 1` (protege `cl`), `bd = 1` (protege la base de datos), `cl = 0` (cantidad de lectores leyendo).
  - 🔢 **Lector**
    1. `down(&mutex)`; `cl = cl + 1`; `if (cl == 1) down(&bd)` — **el primer lector cierra la base** para los escritores; `up(&mutex)`.
    2. Lee la base de datos.
    3. `down(&mutex)`; `cl = cl - 1`; `if (cl == 0) up(&bd)` — **el último lector la abre**; `up(&mutex)`.
  - 🔢 **Escritor** — `down(&bd)`; escribe; `up(&bd)`.
  - 🔑 **El mecanismo en una frase**: los lectores actúan como **un solo bloque**: el primero que entra cierra `bd` y el último que sale la abre. Los del medio pasan gratis: es el patrón del **molinete** (*turnstile*).
  - 🔔 **Separaron "contar" de "bloquear"**: `bd` nunca se usa como contador (vale 0 o 1); el contador es `cl`, una variable común protegida por `mutex`. **`cl` es lo que permite que N lectores compartan UN solo `down`.**

    - 📌 **La "simplificación" de que cada lector haga `down(&bd)` sin `cl` es correcta** (nadie corrompe nada) pero **serializa a los lectores**: entran de a uno, y se pierde la razón de ser del problema. Es el mismo error del mutex de toda la mesa: un candado demasiado grande.

- **La decisión oculta: favorece a los lectores** — mientras haya **al menos un lector** leyendo, los lectores nuevos entran directo y el escritor espera. Si siguen llegando lectores, **el escritor sufre inanición**.
    - 🔢 **Regla verificada**: llega un lector cada **I** segundos y cada lectura dura **D** → **el escritor sufre inanición si I ≤ D** (los lectores se superponen y `cl` nunca llega a 0). En 200 s: I = 2, 4 o 5 con D = 5 → 0 huecos; I = 6 → 33 huecos (el primero en t = 5); I = 10 → 20.
    - **El parche** — cuando llega un escritor, los lectores nuevos esperan detrás de él. Evita la inanición pero **reduce la concurrencia** (y el rendimiento).

- 🌤️ **La regla precisa sobre dormirse con un candado tomado**: *dormirse con un candado tomado es un interbloqueo **SÓLO SI** el que te tiene que despertar necesita **ese mismo** candado.*
    - El lector hace `down(&bd)` **adentro** del mutex y puede dormirse con el mutex tomado: **es seguro**, porque el escritor que lo despierta (`up(&bd)`) **nunca toca `mutex`**.
    - En los filósofos sí sería interbloqueo: F0 y F2 necesitan el mutex para llamar a `probar(1)` (→ U2: La solución buena (fig. 2-46)).
    - En productor-consumidor con los `down` invertidos también: el consumidor necesita el mutex que tiene el productor dormido (→ U2: Productor-consumidor con tres semáforos).
    - ⚠ **Llegué a la conclusión correcta (no es un bug) pero con el razonamiento equivocado.** Lo que lo salva es esta regla, no que "funciona igual".
    - 🔗 Es la puerta a los **interbloqueos de la U6** (espera circular por recursos).

Tema 10 — Sistemas de múltiples procesadores

📖 cap. 8.1, págs. 526–548 (figs. 8-1 a 8-15)

🗒 Repaso U2, pág. 142 — "Tema 10.1 — Hardware de multiprocesador"

- 🌤️ **Todo este tema reusa lo que ya sabés con varias CPUs**: la caché y los buses de la U1, TSL y regiones críticas del tema 6 y 7, y la planificación del tema 8.
    - **?** Mi frase, correcta: *"este tema es lo mismo que toda la exclusión mutua para procesos, pero desde el punto de vista de la CPU"*.

10.1 — Hardware de multiprocesador

📖 cap. 8.1.1, págs. 526–534 (figs. 8-2 a 8-6)

🗒 Repaso U2, pág. 142 — "Tema 10.1 — Hardware de multiprocesador"

- **Multiprocesador con memoria compartida** — dos o más CPUs que **comparten todo el acceso a una RAM común**.
    - La única propiedad rara: una CPU escribe una palabra, la vuelve a leer y **puede obtener otro valor** porque otra CPU la cambió. **No es un defecto: es el canal de comunicación.**
    - 🔗 Es la **condición de carrera elevada a hardware**: en un uniprocesador la causaba el reloj; acá, otra CPU que escribe al mismo tiempo (→ U2: Tema 5 — Condiciones de carrera y regiones críticas).
- **UMA (acceso uniforme a la memoria)** — cada palabra se lee a la **misma velocidad**, esté donde esté.
- **NUMA (acceso no uniforme)** — la memoria **local** es más rápida que la **remota**.
    - 🔢 **Las tres características de NUMA** (el libro las numera)
        1. Hay **un solo espacio de direcciones** visible para todas las CPUs.
        2. El acceso a la memoria remota es con instrucciones **`LOAD` y `STORE`** (no con mensajes).
        3. El acceso a la memoria remota es **más lento** que a la local.
    - **? "Remoto"** = la memoria que está pegada a **otro** nodo; se llega por la red de interconexión, por

eso tarda más.
    - 🔭 *"Todos los programas UMA se ejecutarán sin cambios en las máquinas NUMA, pero el rendimiento será peor."* **NUMA no te obliga a reescribir: te castiga si ubicaste mal los datos.**
    - **NC-NUMA** (sin caché: el costo remoto se ve siempre) contra **CC-NUMA** (con cachés coherentes: se disimula).
    - **Multiprocesadores basados en directorios** — una base de datos en hardware que dice dónde está cada línea de caché y en qué estado. *"Hasta un multiprocesador con memoria compartida realiza muchas operaciones de paso de mensajes de manera interna."* 🔗 (→ U2: Tema 7.4b — Paso de mensajes y barreras).
    - ⚖ **NUMA contra multicomputadora**: comparten solo la característica 3. En NUMA hay un espacio común y `LOAD`/`STORE` (ubicar mal un dato es un problema de **rendimiento**); en la multicomputadora cada nodo tiene memoria privada y hay que usar **`send`/`receive` explícitos** (si no, el programa **no funciona**). 🔭 *NUMA = misma casa, habitación lejana; multicomputadora = otra casa, hay que mandar una carta.*
- **UMA con un solo bus** (fig. 8-2)
    - **(a) Sin caché** — todas pelean por el bus: con 32 o 64 CPUs es imposible, *"la mayoría de las CPUs estarán inactivas la mayor parte del tiempo"*.
    - **(b) Con una caché por CPU** — muchas lecturas se sirven localmente: soporta más CPUs, pero aparece la **coherencia**.
    - **(c) Caché + memoria privada local** — el **compilador** pone ahí el código, las constantes, la pila y las variables locales; la memoria compartida queda solo para lo compartido. Exige cooperación del compilador.
    - 📌 *"Aun con la mejor caché, el uso de un solo bus limita el tamaño de un multiprocesador UMA a 16 o 32 CPUs."* El límite es del **bus único**: con barras cruzadas UMA escala más.
- **Protocolo de coherencia de cachés** — la caché trae **líneas de 32 o 64 bytes**. Un bloque **sólo lectura** puede estar en varias cachés; uno **lectura-escritura** no puede estar en otras. Cuando una CPU escribe, el hardware del bus avisa a las demás cachés:
    - Copia **limpia** → **se DESCARTA** (no se actualiza).
    - Copia **sucia** → **se escribe de vuelta** a memoria o se transfiere directo a la CPU que escribe.
    - ⚠ **Dije que la copia de la otra CPU "se actualiza".** No: **se invalida**. Actualizar haría difundir cada escritura por el bus (se saturaría); invalidar paga solo si el otro vuelve a necesitar el dato. Y además, para escribir hace falta el bloque en modo lectura-escritura, que **exige** que no esté en otras cachés.
    - 🔗 La línea de caché es la unidad que produce el *thrashing* de TSL (→ U2: 10.3 — Sincronización de multiprocesadores); y es la misma caché de la U1 (→ U1: Jerarquía de memoria).
- **Conmutador de barras cruzadas** — una matriz de n CPUs × k memorias con un **punto de cruce** en cada intersección (viene de la telefonía). Defecto: **n × k puntos de cruce**, crece al cuadrado.

10.2 — Los tres tipos de SO multiprocesador

📖 cap. 8.1.2, págs. 534–538 (figs. 8-7, 8-8 y 8-9)

🗐 Repaso U2, pág. 147 — "Tema 10.2 — Los tres tipos de SO multiprocesador"

- 🔭 **La pregunta que resuelven: ¿quién ejecuta el código del SO?** Los tres *"se pueden aplicar de igual forma a los sistemas multinúcleo"*.
- ① **Cada CPU con su propio SO** — n copias del SO, la memoria dividida **estáticamente**; *"las n CPUs operan como n computadoras independientes"*.
    - 🔢 **Los cuatro problemas**
        1. Cada llamada al sistema se atrapa y se maneja **en su propia CPU**.
        2. **No se comparten procesos**: si un usuario se conecta a la CPU 1, todos sus procesos corren en la CPU 1, aunque las otras estén libres.

3. **No se comparten páginas**: una CPU puede quedarse sin memoria mientras otra tiene de sobra.
4. **Cada SO tiene su propia caché de búfer de bloques de disco**: el mismo bloque sucio puede estar en varias cachés → **resultados inconsistentes**. La única salida es eliminar las cachés de búfer, *"lo cual daña el rendimiento en forma considerable"*.

- ○ 💡 **El 2º y el 3º son desperdicio; el 4º es CORRUPCIÓN.** Pasa porque la memoria está dividida pero **el disco es compartido**.
- ○ ⚠ Nombré el síntoma ("se leen cosas desactualizadas") sin el mecanismo: el 4º problema y el disco compartido.

- ② **Maestro-esclavo** — una sola copia del SO, en la **CPU maestra**; todas las llamadas al sistema se redirigen a ella. Resuelve los cuatro problemas con **una sola estructura de datos** (una lista de listos, una reserva de páginas).
  - ○ 📌 **Cuello de botella** — *"si el 10 % del tiempo está manejando llamadas al sistema, con 10 CPUs el maestro casi se saturará y con 20 estará completamente sobrecargado"*.
  - ○ 🔢 **Cuenta**: n.º de CPUs × fracción en llamadas ≤ 1. Ejemplo verificado: 8 CPUs al 15 % → 8 × 0,15 = **1,20** → **saturado**; máximo n ≤ 1 / 0,15 = 6,67 → **6 CPUs** (6 → 90 %; 7 → 105 %).
  - ○ 🔑 **Tiene la misma forma que $\Sigma\, C_i/P_i \leq 1$** de tiempo real (→ U2: Condición de planificabilidad).
- ③ **SMP (multiprocesador simétrico) ★ — una copia del SO en memoria**, y **cualquier CPU la ejecuta**: la CPU que hace la llamada la atiende ella misma.
  - ○ El problema: dos CPUs podrían elegir el mismo proceso o reclamar la misma página. **Deshabilitar interrupciones no sirve** con paralelismo real.
  - ○ **Un mutex único para todo el SO** — correcto, pero *"casi tan mal como el modelo maestro-esclavo"*.
  - ○ **La solución buena** — dividir el SO en **regiones críticas independientes**, cada una con su propio mutex (planificador, sistema de archivos, tabla de páginas…). Hay que cuidar el orden en que se toman para no caer en interbloqueos.
  - ○ 📌 **"SMP resuelve el cuello de botella del maestro-esclavo" es FALSO si el SO tiene un solo candado global.** Lo que lo resuelve es la **granularidad de los candados**.
  - ○ ⚠ Dije que con SMP y un mutex global "la saturación no existe pero deberían esperar su turno": si esperan su turno, la saturación es la misma.
  - ○ 💡 **Candado grande = correcto y lento; candados chicos e independientes = correcto y rápido.** 🔑 Es el `estado[]` de Dijkstra (→ U2: La solución buena (fig. 2-46)).
- 🏷 **Cuadro** — cada CPU su SO: n copias, nada compartido, inconsistencia de cachés de disco. Maestro-esclavo: 1 copia en una CPU, cuello de botella en el maestro. SMP: 1 copia que corren todas, necesita candados finos.

10.3 — Sincronización de multiprocesadores

📖 cap. 8.1.3, págs. 538–542 (figs. 8-10 y 8-11)

🗒 Repaso U2, pág. 152 — "Tema 10.3 — Sincronización de multiprocesadores"

- **Deshabilitar interrupciones "sólo afecta a esa CPU"** — las otras siguen.
- 🔢 **TSL también falla si el bus no se bloquea** (fig. 8-10)
  - ○ La CPU 1 lee el candado y ve 0.
  - ○ Antes de que escriba el 1, la CPU 2 también lee el candado y ve 0.
  - ○ La CPU 1 escribe 1 y entra.
  - ○ La CPU 2 escribe 1 y también entra: **dos en la región crítica**.
  - ○ 💡 **La indivisibilidad tiene que ser del BUS, no de la instrucción.** La solución: la CPU **bloquea el bus** activando una línea especial mientras hace el TSL.
  - ○ 🔑 **Por eso se inventó Peterson (1981)**: para sincronizar **solo con software**, sin depender de un bus bloqueable (→ U2: ✓ Solución 4 — Peterson).

- **El thrashing de caché (compartición falsa)** — TSL **escribe**, así que exige el bloque en modo exclusivo: invalida la copia del dueño del candado; el dueño toca una palabra vecina **del mismo bloque** y lo reclama; el bloque viaja de caché en caché (**ping-pong**) y genera **más tráfico que leer la palabra suelta**.
- **Los tres arreglos, en orden de calidad**
  - **Lectura pura antes del TSL** — `while (candado != 0) ;` y recién cuando ve 0, `if (TSL(&candado) == 0) break;`. Mientras lee, todas comparten el bloque **sólo lectura** y no hay tráfico.
    - 📌 **Si dos ven libre a la vez no es una condición de carrera**: *"la verdadera adquisición la realiza la instrucción TSL, y esta instrucción es atómica"*; *"el éxito en la lectura pura es tan sólo una sugerencia"*.
  - **Backoff exponencial binario de Ethernet** (Anderson, 1990) — entre intentos se espera 1, 2, 4, 8… instrucciones, hasta un máximo. Máximo bajo → más tráfico; máximo alto → el candado queda libre sin que nadie lo tome.
  - **Candado privado por CPU** (Mellor-Crummey y Scott, 1991; fig. 8-11) — cada CPU que espera gira sobre **su propia variable**, en un bloque de caché no usado, y se encola en una **lista FIFO**; al liberar, el dueño le pasa el candado al primero de la lista. **Cero tráfico de bus** y **libre de inanición**.
  - 🔗 La lista FIFO del MCS es la **cola de bloqueados del semáforo**, pero girando (→ U2: Tema 7.2 — Semáforos).
- **Espera activa contra conmutación** — la CPU quiere un mutex tomado: ¿**gira** o **conmuta** a otro hilo?
  - 📌 **A veces no hay opción**: el candado de la **lista de procesos listos** obliga a girar, *"ya que para ello tendría que leer la lista de procesos listos"*. **No se puede conmutar para poder conmutar.**
  - La pregunta **no existe en un uniprocesador** (ahí girar nunca sirve: el que tiene el candado no puede avanzar mientras vos girás).
  - **Costo de conmutar** = dos cambios de contexto **+ la caché y la TLB frías**.
  - 🔢 **Ejemplo del libro** — mutex de **50 µs** con conmutación de 1 ms + 1 ms → **girar**; mutex de **10 ms** → **conmutar**.
  - 🔢 **Ejemplo verificado** — conmutar cuesta 1,5 + 1,5 = 3000 µs; candado de 400 µs → **girar**; candado de 20 ms → la cuenta dice conmutar, pero **no se puede** si es la lista de listos.
  - **Tres diseños** — girar siempre / conmutar siempre / decidir cada vez.
  - **Algoritmo de retrospección** — saber de antemano cuánto va a durar el candado: es el punto de referencia **irrealizable** (el mismo papel que SJF y el reemplazo óptimo). 🔗 (→ U2: SJF (el trabajo más corto primero)).
  - **Umbral dinámico** (Karlin y colaboradores, 1991) — se decide con la **historia observada**: si las 3 esperas previas fueron de ~200 µs, girar hasta 2 ms; si las 3 llegaron a los 2 ms, **conmutar de inmediato**. 🔗 **Es el promedio exponencial del SPN aplicado a candados** (→ U2: El proceso más corto a continuación (SPN)).

10.4 — Planificación de multiprocesadores

📖 cap. 8.1.4, págs. 542–548 (figs. 8-12 a 8-15)

🗒 Repaso U2, pág. 159 — "Tema 10.4 — Planificación de multiprocesadores"

- 📌 **Ahora hay DOS dimensiones**: *qué hilo ejecutar* **y** *en cuál CPU*. En un uniprocesador había una sola. Es la respuesta de una línea a "¿qué cambia en la planificación?".
- **¿Procesos o hilos?** — con hilos de usuario se planifica por proceso; con hilos de kernel, el kernel elige el hilo y el proceso *"sólo tiene un pequeño papel (o tal vez ninguno)"* 🔗 (→ U2: 8.6 — Planificación de hilos).
- **Hilos no relacionados** (usuarios independientes, tiempo compartido) contra **relacionados** (trabajan juntos y se comunican).
  - 🔔 **El `make` paralelo**: *"si se necesitan 10 compilaciones, no tiene caso programar 9 de ellas de inmediato y dejar la última para mucho después"*. Cuando el resultado depende del **último** hilo, hay

que optimizar el **máximo**, no el promedio. 🔗 Misma lógica que la **barrera** (→ U2: Barreras).

- **A) Tiempo compartido — una sola cola global** (fig. 8-12: 16 CPUs, 14 hilos en listas de prioridad 7, 5, 4, 2, 1, 0)
  - 🔢 La CPU 4 queda libre → **bloquea las colas** → toma el de mayor prioridad (A). Después la CPU 12 queda libre → toma B.
  - Ventajas: reparto como en un uniprocesador y **balance automático de carga** (*"no puede haber una CPU inactiva mientras otras están sobrecargadas"*).
  - 🔢 **Las dos desventajas**: 1 la **contención** por la estructura de datos al crecer las CPUs; 2 la **sobrecarga del cambio de contexto** cuando un hilo se bloquea.
  - 🔗 La cola global es la región crítica de la **lista de listos**, la que obliga a girar (→ U2: Espera activa contra conmutación).
- **La anomalía del spin lock** — un hilo con un candado de espera activa **pierde la CPU por fin de quantum**: las otras CPUs giran en vano hasta que se lo vuelva a planificar.
  - **Planificación inteligente** (Zahorjan y colaboradores, 1991) — el hilo que toma un spin lock pone una **bandera a nivel de proceso** (y la borra al soltarlo); el planificador **no lo detiene**, *"le da un poco más de tiempo para que complete su región crítica"*.
  - 🔗 **Es la inversión de prioridades con varias CPU** (→ U2: Inversión de prioridades): la bandera es justo la información que en el tema 8 el planificador "no tenía". Primo barato de la herencia de prioridades.
- **Afinidad de caché** (Vaswani y Zahorjan, 1991) — *"ciertas CPUs son más iguales que otras"*: si A corrió mucho en la CPU k, la caché (y la TLB) de k está llena de A → conviene volver a correrlo en k.
- **Planificación de dos niveles**
  - **Nivel superior** — al **crear** el hilo se lo asigna a una CPU (por ejemplo, la de menor carga); cada CPU tiene su propia colección.
  - **Nivel inferior** — cada CPU planifica **sus** hilos por separado.
  - Si una CPU se queda sin hilos, **toma uno de otra CPU** en vez de quedar inactiva (robo de trabajo). El hilo robado llega con la caché fría, pero una CPU ociosa pierde más.
  - 🔢 **Los tres beneficios**
    1. Distribuye la carga casi uniformemente.
    2. Aprovecha la **afinidad de caché**.
    3. **Minimiza la contención** por las listas de listos (cada CPU tiene la suya).
  - 🔗 El 3º es otra vez partir un candado grande en candados chicos (→ U2: 10.2 — Los tres tipos de SO multiprocesador).
- **B) Espacio compartido** (fig. 8-13: 32 CPUs en particiones de 4, 6, 8 y 12, con 2 libres) — al crear un grupo de N hilos relacionados, si hay N CPUs libres cada uno recibe **su CPU dedicada, sin multiprogramación**; si no, **ninguno arranca**.
  - Ventaja: **sin sobrecarga** de cambio de contexto. Defecto: **un hilo bloqueado retiene su CPU, que queda inactiva**.
  - El análogo de SJF es minimizar **cuenta de CPUs × tiempo de ejecución**, pero el dato rara vez está disponible: *"es difícil vencer al algoritmo primero en llegar, primero en ser atendido"* (Krueger y colaboradores, 1994).
  - **Servidor central** (Tucker y Gupta, 1989) — cada aplicación sondea cuántas CPUs puede usar y ajusta su número de hilos (el servidor web que baja de 10 a 5). 🔗 Política contra mecanismo (→ U2: 8.5 — Política contra mecanismo).
- **C) Planificación por pandillas** ★ (Ousterhout, 1982; sale de la **co-planificación**)
  - **El problema (fig. 8-14)** — A0 y B0 comparten la CPU 0, A1 y B1 la CPU 1, y A0 con A1 se comunican: están **fuera de fase**, así que hay **una secuencia solicitud-respuesta cada 200 ms**. Las CPU están ocupadas con **trabajo útil de B**: el problema es la **latencia** de A, no la utilización.
  - 🔢 **Las tres partes**

1. Los grupos de hilos relacionados se planifican **como una unidad (pandilla)**.
2. Todos los miembros de la pandilla se ejecutan **simultáneamente** en distintas CPUs de tiempo compartido.
3. Todos los miembros **inician y terminan sus intervalos en conjunto**.

- **El truco** — el tiempo se divide en **quantums discretos**; al inicio de cada uno **todas las CPUs se reprograman a la vez** (la edición en castellano dice "asíncrona", pero el mecanismo es sincrónico); dentro del quantum no se planifica nada; **si un hilo se bloquea, su CPU queda inactiva hasta el fin del quantum**.
- 🔢 **Fig. 8-15** (6 CPUs, 5 procesos, 24 hilos) — ranura 0: A0–A5; ranura 1: B0, B1, B2, C0, C1, C2; ranura 2: D0–D4 y E0; ranura 3: E1–E6; la ranura 4 repite la 0.
- **Costo** — los huecos: una CPU inactiva hasta el fin del quantum cuando un hilo se bloquea. **No conviene con hilos no relacionados** (no ganan nada y pagan los huecos) ni con aplicaciones muy limitadas a E/S.
- ⚠ **En la respuesta del 10.4 escribí que al terminar el quantum el hilo "se va a dormir"** (va a LISTO) y que las CPU estaban al 100 % "por cambios de contexto inútiles" (hacen trabajo útil de otros hilos).

- 🏷 **Los tres enfoques** — tiempo compartido: hilos no relacionados, reparte en el tiempo, la CPU de un hilo bloqueado se le da a otro, problema: la contención. Espacio compartido: relacionados, reparte en el espacio, sin multiprogramación, la CPU queda inactiva. Pandillas: relacionados, tiempo y espacio, multiprogramación sincronizada, inactiva hasta el fin del quantum.
  - 💡 *Tiempo compartido = cada uno por su lado; espacio compartido = cada grupo su cancha; pandillas = todos a la vez, al mismo paso.*

Cierre de la U2 y lo que más se toma

🗐 Repaso U2, pág. 167 — "Cierre de la Unidad 2"

- **Los 8 puntos**
  1. **Proceso ≠ hilo**: el proceso agrupa, el hilo ejecuta; `fork` duplica, `execve` reemplaza; n fork = 2^n (→ U2: Tema 1 — Qué es un proceso).
  2. **Tres estados y cuatro transiciones**; fin de quantum → LISTO; no existe BLOQUEADO → EJECUCIÓN (→ U2: Las 4 transiciones y quién provoca cada una).
  3. **Las 4 condiciones** y cuál viola cada intento: interrupciones → 2, candado → 1, alternancia → 3, Peterson y TSL funcionan (→ U2: Las 4 condiciones de una buena solución).
  4. **Espera ocupada contra bloqueo**; la inversión de prioridades es culpa de la espera ocupada (→ U2: Inversión de prioridades).
  5. **Semáforos**: un `up` sin destinatario se guarda; `wakeup` y `signal` se pierden; productor-consumidor con tres semáforos y el orden de los `down` (→ U2: Productor-consumidor con tres semáforos).
  6. ★ **Planificación**: retorno, espera y respuesta; los algoritmos; $\Sigma\ C_i/P_i \leq 1$ (→ U2: Las métricas).
  7. **Problemas clásicos**: Dijkstra deja comer a 2 como máximo; escritor con inanición si $I \leq D$; la regla de dormirse con el candado (→ U2: La regla precisa sobre dormirse con un candado tomado).
  8. **Multiprocesadores**: candado grande contra candados chicos; dos dimensiones; pandillas (→ U2: 10.4 — Planificación de multiprocesadores).
- **Lo que más se toma** (fotos del Integrador 22/11/2024 y el Final 17/12/2025)
  - **Round Robin con quantum 1 y 2**: Gantt, métricas y **la cola de preparados en un instante t**, con el desempate de la cátedra (→ U2: 8.3 — Round Robin).
  - **SJN y SRTF/SRTN** (→ U2: 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF).
  - Comandos: `ps -ef`, `pgrep`, `kill -STOP` / `kill -CONT`, `strace` (→ U1: strace).
  - Formato típico: 5 a 7 consignas, 60 minutos, se aprueba con 60 puntos.
- **Simuladores de la unidad** — `planificacion_cpu.html` (Gantt con la cola de preparados y los ejercicios de la cátedra), `filosofos_comelones.html`, `productor_consumidor.html`,

`condicion_de_carrera.html`, `estados_de_proceso.html` (todos en `unidades/u02-procesos/visuales/`).

Anexo A — Todos los errores que ya cometí (⚠)

- U1 — Introducción, hardware y estructura de los SO › Tema 1 — ¿Qué es un sistema operativo?
  ⚠ **El disco tiene dos recursos adentro**: la superficie se reparte en el **espacio**, pero el **brazo**, que es uno solo, se reparte en el **tiempo** (de ahí sale la planificación de disco de la U5). En el cuestionario me faltó identificar qué se multiplexa de cada forma (parte B-a, 1,5/2).
- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware
  ⚠ Error de la clase: decir que "el SO se cuelga y después reacciona". No: el hardware la frena antes.
- U1 — Introducción, hardware y estructura de los SO › Tema 4 — El arranque y el sector de booteo
  ⚠ **La tabla no contiene las particiones: las describe.** Una partición de 50 MB o de 2 TB ocupa igual 16 bytes. (Error que cometí: pensar que los 64 bytes "eran" las particiones.)
- U1 — Introducción, hardware y estructura de los SO › Tema 4 — El arranque y el sector de booteo
  ⚠ Los confundí en el cuestionario (parte B-d, 1,5/2).
- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema
  ⚠ **Un buffer más grande NO reduce la lectura de disco: reduce las llamadas al sistema.** (Error del cuestionario, parte B-e, 0/2.)
- U1 — Introducción, hardware y estructura de los SO › Tema 9 — Estructuras de los sistemas operativos
  ⚠ **Las capas NO dan tolerancia a fallas**: todo sigue en modo kernel, y un driver que falla tumba el sistema igual que en la monolítica. Lo que mejora es la **claridad para depurar**. (Error de la clase y del cuestionario, parte B-b, 0/2.)
- U1 — Introducción, hardware y estructura de los SO › Tema 9 — Estructuras de los sistemas operativos
  ⚠ **No confundir la estructura del SO con la arquitectura de una aplicación** (tu app que le manda SQL a un motor). Y no elegir cliente-servidor para algo sensible a la latencia: es el más lento.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso
  ⚠ **Contesté 4 para tres fork; son 8** (7 hijos + el original). El fork no suma uno: multiplica por dos.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  ⚠ **Dije que "el proceso bloqueado hace una interrupción"**. No: un proceso bloqueado no puede hacer nada. La interrupción la genera **el controlador del dispositivo** (hardware) (→ U1: Tratamiento de una interrupción por hardware).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  ⚠ **Fin de quantum → LISTO, nunca BLOQUEADO.** Lo decide el planificador. Pasar a BLOQUEADO lo decide el proceso, pidiendo algo que no tiene. (Error que repetí en la U2 dos veces: en la inversión de prioridades y en la respuesta del 10.4.)
- U2 — Procesos, hilos, concurrencia y planificación › Tema 4 — Implementación de hilos
  ⚠ **Atribuí a "otro espacio de memoria" que un hilo egoísta no afecte a otros procesos.** La razón correcta: **el kernel sí tiene interrupciones de reloj** (para los procesos).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 4 — Implementación de hilos
  ⚠ **Mi objeción, correcta**: con 1 hilo de kernel es igual a hilos de usuario puros (**N:1**), y con tantos hilos de kernel como de usuario es igual a hilos de kernel puros (**1:1**). Esos son los **extremos**; la híbrida es **M:N**, el punto intermedio.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  ⚠ **Dije que violaba la condición 2; viola la 3.** El que bloquea no estaba usando la región crítica.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  ⚠ **Creía que "el de mayor prioridad nunca llega a ejecutarse".** Es al revés: **H consigue la CPU siempre; lo que no consigue es el candado.** La prioridad no falla: se respeta de más.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  ⚠ **Creía que al acabarse el quantum H "pasa a bloqueado".** No: fin de quantum → **LISTO**, y como H es el más prioritario el planificador **lo vuelve a elegir**. Con espera ocupada **ninguna de las 4 instrucciones de TSL es una llamada al sistema**, así que **H no tiene con qué bloquearse**.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada

⚠ **Tendí a culpar a TSL.** TSL garantiza exclusión mutua perfecta: **el culpable es la espera ocupada**, que le pasa igual a Peterson, a XCHG y a la variable candado.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos

  ⚠ **Pensaba `up` y `down` como "sumar" y "restar"**, en vez de "entregar" y "pedir". **No hay forma de esperar con un `up`**, así que un `up` nunca puede servir para pedir lugar. **`vacias` y `llenas` cuentan recursos, no eventos.**

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos

  ⚠ Vocabulario: dije *"las llamadas perdidas acumuladas"*. Si se acumulan **no están perdidas**: están **guardadas**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos

  ? ⚠ **"Los semáforos usan TSL para bloquear el bus y que no haya interrupciones"** mezcla dos cosas: **deshabilitar interrupciones** protege en **una** CPU; **TSL** bloquea el **bus** para las **otras** CPU. Se usan las dos, cada una para lo suyo.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos

  ⚠ El interbloqueo lo expliqué bien, pero dije que los dos quedaban "ocupando tiempo de CPU". **Bloqueado = fuera de la cola de listos: no consume ni un ciclo.**

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4a — Monitores

  ⚠ **Invertí esa asimetría**: dije que con semáforos también habría que condicionar el `up`. Es al revés. Después la reformulé bien: *"el monitor se fija si hay alguien dormido; en el semáforo depende de los números"*.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4a — Monitores

  ⚠ **Creí que una señal perdida es siempre un error.** No: **señal perdida con alguien esperándola = desastre** (el wakeup perdido); **sin nadie esperando = trabajo al vacío**. El `if cuenta = 1` es una **optimización**, no una condición de correctitud.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4b — Paso de mensajes y barreras

  ⚠ **Creí que exige procesos separados** y que no se puede usar entre hilos ni en una sola máquina. Al revés: **no NECESITA memoria compartida, pero funciona igual si la hay**; es el mecanismo **más general**. Si no se usa con memoria compartida es por el **costo de copiar**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas

  ⚠ **Cuando clasifiqué A (2 ms de CPU + 50 ms de disco) usé la E/S para justificar.** La columna de E/S no se mira nunca: es el distractor.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas

  ⚠ El compañero que decía "la espera no puede ser 10 si esperó 12 como máximo" confundía retorno con espera: P3 llegó en 2, no en 0.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3b — Planificación por prioridades y múltiples colas

  ⚠ **Convención**: con `1/f`, **el número más grande es el más prioritario**, al revés de la cátedra (**1 = más prioritario**). **Declarar siempre la convención en la respuesta.** Lo que no cambia nunca: *poca CPU → favorecer; mucha CPU → castigar.*

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3b — Planificación por prioridades y múltiples colas

  ⚠ El ejemplo de prioridades dinámicas del otro chat usa "número mayor = más prioridad" y una regla de ±1 por tic que **no está en el libro**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativ

  ⚠ **Leí los boletos como porcentajes sin dividir por el total.** Siempre: porcentaje = boletos / total.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativ

  ⚠ La explicación del otro chat ("B con prioridad media 15 se metería en el medio") confunde boletos con números de prioridad.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.6 — Planificación de hilos

  ⚠ Justifiqué la elección de hilos de usuario con "puede ceder fácilmente"; lo que decide es **el precio de cada conmutación**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.1 — Los filósofos comelones

  ⚠ **Contesté que no funcionaba**, porque confundí cuál es el tenedor derecho de F4: es el **tenedor 0** = (4 + 1) % 5, no el 4. F0 y F4 pelean por el 0, el tenedor 4 queda libre y F3 puede comer. Sí funciona: verificado con búsqueda exhaustiva de todos los estados.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  ⚠ **Lo que lo diferencia de todo lo anterior**: no todos los accesos son iguales. Exclusión mutua solo **entre escritores** y **entre un escritor y los lectores**.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  ⚠ **Llegué a la conclusión correcta (no es un bug) pero con el razonamiento equivocado.** Lo que lo salva es esta regla, no que "funciona igual".
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.1 — Hardware de multiprocesador

  ⚠ **Dije que la copia de la otra CPU "se actualiza".** No: **se invalida**. Actualizar haría difundir cada escritura por el bus (se saturaría); invalidar paga solo si el otro vuelve a necesitar el dato. Y además, para escribir hace falta el bloque en modo lectura-escritura, que **exige** que no esté en otras cachés.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres tipos de SO multiprocesador

  ⚠ Nombré el síntoma ("se leen cosas desactualizadas") sin el mecanismo: el 4º problema y el disco compartido.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres tipos de SO multiprocesador

  ⚠ Dije que con SMP y un mutex global "la saturación no existe pero deberían esperar su turno": si esperan su turno, la saturación es la misma.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  ⚠ **En la respuesta del 10.4 escribí que al terminar el quantum el hilo "se va a dormir"** (va a LISTO) y que las CPU estaban al 100 % "por cambios de contexto inútiles" (hacen trabajo útil de otros hilos).

Anexo B — Todas las conexiones entre unidades (🔗)

- La idea que une todo

  Monolítico contra microkernel (→ U1: Tema 9 — Estructuras de los sistemas operativos).
- La idea que une todo

  Hilos contra procesos (→ U2: Tema 3 — Hilos: el modelo clásico).
- La idea que une todo

  Memoria compartida contra paso de mensajes (→ U2: Tema 7.4b — Paso de mensajes y barreras).
- La idea que une todo

  El mutex de toda la mesa de los filósofos contra el `estado[]` de Dijkstra (→ U2: 9.1 — Los filósofos comelones).
- La idea que une todo

  SMP con un mutex global contra SMP con un mutex por subsistema (→ U2: 10.2 — Los tres tipos de SO multiprocesador).
- La idea que une todo

  Una cola global de listos contra una cola por CPU (→ U2: 10.4 — Planificación de multiprocesadores).
- La idea que une todo

  📡 **Cuando no podés avanzar, ¿en qué estado te quedás?** Girar = EN EJECUCIÓN (gasta CPU), ceder = LISTO, dormir = BLOQUEADO (no gasta CPU). Es la pregunta de los temas 6 y 7 de la U2 (→ U2: Los tres niveles de espera).
- La idea que une todo

💡 **El algoritmo óptimo siempre necesita un dato del futuro.** SJF necesita las ráfagas (→ U2: 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF), el algoritmo de retrospección necesita saber cuánto dura el candado (→ U2: 10.3 — Sincronización de multiprocesadores), y en la U3 el reemplazo óptimo de páginas necesita las referencias futuras.

- La idea que une todo

  💡 **Mecanismo contra política.** El mecanismo dice *cómo* se puede hacer algo; la política dice *quién y cuándo*. Aparece en el microkernel (→ U1: Tema 9 — Estructuras de los sistemas operativos) y en la planificación (→ U2: 8.5 — Política contra mecanismo).

- U1 — Introducción, hardware y estructura de los SO › Tema 1 — ¿Qué es un sistema operativo?

  Multiplexaje en el TIEMPO — los recursos se turnan (CPU, impresora). Genera el problema de *¿quién va primero?* → 🔗 planificación (→ U2: Tema 8 — Planificación de la CPU ★).

- U1 — Introducción, hardware y estructura de los SO › Tema 1 — ¿Qué es un sistema operativo?

  🔗 Es la justificación numérica de por qué existe la multiprogramación (→ U1: Tema 7 — Historia por generaciones) y por qué el planificador elige entre los LISTOS (→ U2: Tema 2 — Estados del proceso y el BCP).

- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware

  ⚖ **PSW contra campo "estado" del BCP**: el PSW es un **registro del hardware** (bits de condición y modo). El estado del BCP es un **dato del SO** (en ejecución, listo o bloqueado). Son cosas distintas que comparten la palabra "estado" (→ U2: BCP (bloque de control de proceso)).

- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware

  TRAP — la única puerta legal para pasar de usuario a kernel (→ U1: Tema 6 — Llamadas al sistema).

- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware

  🔗 Sin estos dos modos no habría multiprogramación segura (→ U1: Tema 7 — Historia por generaciones) ni estructuras de SO (→ U1: Tema 9 — Estructuras de los sistemas operativos).

- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware

  🔗 La TLB de la U3 es una caché de traducciones: misma idea, mismo cálculo de tiempo medio. Y la caché vuelve en los multiprocesadores, con la coherencia (→ U2: 10.1 — Hardware de multiprocesador).

- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware

  ? El BIOS no está en la RAM ni en el disco: está en un chip ROM/flash de la placa madre. Por eso puede arrancar sin disco (→ U1: Tema 4 — El arranque y el sector de booteo).

- U1 — Introducción, hardware y estructura de los SO › Tema 3 — Las tres formas de hacer E/S

  🔗 **Es la misma espera ocupada de la U2**: gastar CPU girando hasta que pase algo (→ U2: Tema 6 — Exclusión mutua con espera ocupada).

- U1 — Introducción, hardware y estructura de los SO › Tema 3 — Las tres formas de hacer E/S

  Deshabilitar interrupciones — la CPU puede postergarlas; el dispositivo sigue señalizando, pero la CPU no se interrumpe hasta que se rehabilitan. 🔗 En la U2 es el primer intento de exclusión mutua (→ U2: Tema 6 — Exclusión mutua con espera ocupada).

- U1 — Introducción, hardware y estructura de los SO › Tema 3 — Las tres formas de hacer E/S

  🔗 La versión de la U2 de esta secuencia, con el BCP y el planificador, son los 8 pasos de la fig. 2-5 (→ U2: Manejo de una interrupción y cambio de contexto).

- U1 — Introducción, hardware y estructura de los SO › Tema 5 — Ensamblador e interrupciones del 8086

  🔗 00h **bloquea**; 01h **sondea** (polling): es el mismo par interrupción/espera ocupada del tema 3 (→ U1: Tema 3 — Las tres formas de hacer E/S).

- U1 — Introducción, hardware y estructura de los SO › Tema 5 — Ensamblador e interrupciones del 8086

  💡 **INT se parece a un CALL**: apila la dirección de retorno y el PSW, salta a la rutina y la rutina termina con **IRET**, que desapila todo. Es un CALL con control de seguridad. 🔗 Es el antecedente de TRAP (→ U1: Tema 6 — Llamadas al sistema).

- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema

  CALL, RET y la pila — el procedimiento no sabe quién lo llamó: el CALL **apila** la dirección de la línea siguiente y salta; el RET **desapila** esa dirección y vuelve. Las llamadas anidadas se desapilan en orden inverso (LIFO). 🔗 Por eso cada hilo necesita su propia pila (→ U2: La pila).

- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema

  📌 **La llamada puede bloquear al proceso**: si se lee del teclado y no hay nada, el proceso se bloquea, el

SO corre otro, y los pasos 9 a 11 se completan cuando llega la entrada. 🔗 Transición ① del diagrama de estados (→ U2: Tema 2 — Estados del proceso y el BCP).

- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema

  Procesos — `fork` (crea un hijo idéntico) 🔗 (→ U2: Tema 1 — Qué es un proceso); `execve` (reemplaza la imagen); `exit` (termina); `waitpid` (espera a un hijo); `kill` (envía una señal).
- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema

  ? **`close(fd)`** — libera el descriptor: el número deja de servir. 🔗 Con hilos, lo pierden todos los hilos del proceso (→ U2: Tema 3 — Hilos: el modelo clásico).
- U1 — Introducción, hardware y estructura de los SO › Tema 7 — Historia por generaciones

  Agujero 3 (1965–1980, circuitos integrados): la CPU espera al disco — en datos comerciales la CPU esperaba E/S el 80 a 90 % del tiempo. Solución: **multiprogramación** (varios trabajos en memoria a la vez; cuando uno espera el disco, la CPU pasa a otro). 🔗 El modelo $1 - p^n$ (→ U1: Modelo de la multiprogramación).
- U1 — Introducción, hardware y estructura de los SO › Tema 7 — Historia por generaciones

  📌 **La multiprogramación exige hardware de protección** (si no, un trabajo pisa la memoria de otro). Recién lo trajo la tercera generación (IBM 360). 🔗 (→ U1: Modo kernel y modo usuario).
- U1 — Introducción, hardware y estructura de los SO › Tema 7 — Historia por generaciones

  Tiempo compartido — cada usuario con su terminal; si de 20 conectados 17 están pensando, la CPU atiende por turnos a los 3 que piden. Primer sistema: **CTSS** (MIT). 🔗 Es el origen de Round Robin y de la planificación interactiva (→ U2: 8.3 — Round Robin).
- U1 — Introducción, hardware y estructura de los SO › Tema 8 — Tipos de sistemas operativos

  De multiprocesadores — varias CPU en un sistema. 🔗 Desarrollado en la U2, tema 10 (→ U2: Tema 10 — Sistemas de múltiples procesadores).
- U1 — Introducción, hardware y estructura de los SO › Tema 8 — Tipos de sistemas operativos

  🔗 En la U2 aparece la condición de planificabilidad $\Sigma\, C_i/P_i \leq 1$ (→ U2: 8.4 — Planificación de tiempo real).
- U1 — Introducción, hardware y estructura de los SO › Tema 9 — Estructuras de los sistemas operativos

  Mecanismo en el kernel, política en el espacio de usuario. 🔗 La misma regla en planificación (→ U2: 8.5 — Política contra mecanismo).
- U1 — Introducción, hardware y estructura de los SO › Tema 9 — Estructuras de los sistemas operativos

  4. Cliente-servidor — variación del microkernel: servidores y clientes comunicados por **paso de mensajes**; el cliente no sabe si el servidor está en la misma máquina o en la red. Base de los sistemas distribuidos (U8). 🔗 (→ U2: Tema 7.4b — Paso de mensajes y barreras).
- U1 — Introducción, hardware y estructura de los SO › Tema 9 — Estructuras de los sistemas operativos

  🔗 **Es el mismo intercambio que hilos contra procesos**: Chrome eligió "procesos" como quien elige microkernel (→ U2: Tema 3 — Hilos: el modelo clásico).
- U1 — Introducción, hardware y estructura de los SO › Lo que más se toma de la U1

  `strace -o salida.txt ls -la` (→ U1: strace).
- U1 — Introducción, hardware y estructura de los SO › Lo que más se toma de la U1

  INT 21h: completar el código para mostrar una cadena (AH = 09h, cadena con $, DS:DX) (→ U1: INT 21h (DOS), la más importante).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso

  📌 50 procesos en un solo núcleo es un sistema **concurrente**, no **paralelo**. 🔗 El paralelismo real es el tema 10 (→ U2: Tema 10 — Sistemas de múltiples procesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso

  ? El evento 2 quiere decir que un proceso que ya está corriendo ejecuta `fork` (que es una llamada al sistema, o sea un TRAP) 🔗 (→ U1: TRAP).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP

  ⚠ **Dije que "el proceso bloqueado hace una interrupción"**. No: un proceso bloqueado no puede hacer nada. La interrupción la genera **el controlador del dispositivo** (hardware) (→ U1: Tratamiento de una interrupción por hardware).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP

  ③ El planificador elige a Q (que estaba LISTO). La CPU no queda ociosa: **eso es la multiprogramación**

🔗 (→ U1: Tema 7 — Historia por generaciones).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP

  🔗 Los tres primeros campos (registros, PC, PSW) son los de la U1: el SO tiene que conocerlos **para guardarlos y restaurarlos** (→ U1: La CPU y sus registros).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP

  ? El BCP es del proceso; cada hilo tiene además su propia entrada en una tabla de hilos con su PC, registros, pila y estado. El diagrama de estados vale igual para los hilos (→ U2: Tema 3 — Hilos: el modelo clásico).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP

  El cambio de contexto no es gratis: pasar a modo kernel, guardar y restaurar registros, cambiar el mapa de memoria y muchas veces **invalidar la caché**. 🔗 Es lo que hace que el tamaño del quantum sea un problema real (→ U2: 8.3 — Round Robin).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico

  La pila — zona de memoria **LIFO** (el último que entra es el primero que sale). Por cada procedimiento activo guarda un **marco de pila**: la **dirección de retorno**, los **parámetros** y las **variables locales**. El registro SP apunta al tope (→ U1: La CPU y sus registros).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico

  📌 **Por qué cada hilo necesita su propia pila**: cada uno tiene un historial de llamadas distinto. Con una pila compartida, el `return` de un hilo desapilaría el marco del otro. 🔗 Es la pila del CALL/RET de la U1 (→ U1: Tema 6 — Llamadas al sistema).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico

  🌤 **Compartir ≠ coordinarse**: compartir lo hace el sistema gratis; coordinarse lo escribe el programador. Compartir sin coordinarse produce **condiciones de carrera** (→ U2: Tema 5 — Condiciones de carrera y regiones críticas).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico

  🌤 **`thread_yield` existe porque dentro de un proceso no hay interrupciones de reloj**: los hilos de usuario tienen que cederse la CPU voluntariamente (→ U2: Tema 4 — Implementación de hilos).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico

  🔗 **Es el mismo intercambio que monolítico contra microkernel** (→ U1: Tema 9 — Estructuras de los sistemas operativos), y vuelve con memoria compartida contra paso de mensajes (→ U2: Tema 7.4b — Paso de mensajes y barreras).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 4 — Implementación de hilos

  🔗 **La planificación de estos dos tipos de hilos** es el tema 8.6 (→ U2: 8.6 — Planificación de hilos).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 5 — Condiciones de carrera y regiones críticas

  ? Si al que está en la región crítica se le acaba el quantum: pasa a LISTO **sin salir** de la región; los que quieren entrar siguen sin poder (esperan o se bloquean), y los que no necesitan el recurso se ejecutan normalmente. **Perder la CPU no es salir de la región crítica.** 🔗 Es la semilla de la inversión de prioridades (→ U2: Inversión de prioridades).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada

  🔗 Es la misma idea de la E/S programada de la U1: sondear sin parar (→ U1: 1. E/S programada (espera ocupada, polling)).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada

  🔗 **Peterson se inventó para sincronizar completamente por software**; TSL necesita un bus que se pueda bloquear (→ U2: 10.3 — Sincronización de multiprocesadores).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada

  📌 **"Girar 1000 veces y después dormir un rato"** no lo resuelve del todo: no hay cómo elegir cuánto dormir sin suponer velocidades; se pierde la garantía de prioridad; se gastan igual 1000 vueltas; y con un tercer proceso M de prioridad media se vuelve a colgar (**dormir garantiza que H suelte la CPU, no que L la reciba**). 🌤 **Dormir por tiempo ≠ dormir por evento**; lo segundo es el semáforo (→ U2: Tema 7.2 — Semáforos).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada

  🔗 **La misma anomalía vuelve en los multiprocesadores** (el spin lock al vencer el quantum) y se resuelve con la planificación inteligente (→ U2: 10.4 — Planificación de multiprocesadores).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  Apropiativa contra no apropiativa (adelanto del tema 8) — **no apropiativa**: el proceso suelta la CPU solo cuando se bloquea o termina; **apropiativa**: el sistema se la puede sacar (por ejemplo, con la interrupción de reloj). 🖊 (→ U2: 8.1 — Conceptos y métricas).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  🖊 **Regla general que sale de acá**: *nunca te quedes inactivo —ni girando ni dormido— con un recurso tomado que otro necesita para desbloquearte* (→ U2: La regla precisa sobre dormirse con un candado tomado).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos
  🖊 Es el primer caso de la regla del candado (→ U2: La regla precisa sobre dormirse con un candado tomado).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.3 — Mutexes
  🔅 Son tres estados distintos del diagrama del tema 2. **Toda la unidad es la misma pregunta: cuando no podés avanzar, ¿en qué estado te quedás?** (→ U2: Tema 2 — Estados del proceso y el BCP)
- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4b — Paso de mensajes y barreras
  🖊 **El grupo avanza al ritmo del más lento**: la misma lógica del `make` paralelo y de las pandillas (→ U2: 10.4 — Planificación de multiprocesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4b — Paso de mensajes y barreras
  ⚖ **Memoria compartida contra paso de mensajes** — compartir es rápido pero peligroso; aislar es seguro pero caro. 🖊 (→ U2: Tema 3 — Hilos: el modelo clásico), (→ U1: Tema 9 — Estructuras de los sistemas operativos) y los multiprocesadores basados en directorios, que internamente pasan mensajes (→ U2: 10.1 — Hardware de multiprocesador).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas
  Planificador — la parte del SO que elige qué proceso se ejecuta; usa un **algoritmo de planificación**. **Solo elige entre los LISTOS** (→ U2: Tema 2 — Estados del proceso y el BCP).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas
  Conmutar no es gratis — pasar a modo kernel, guardar el estado en el BCP, cambiar el mapa de memoria e **invalidar la caché** (→ U2: Manejo de una interrupción y cambio de contexto). Por eso no se puede conmutar todo el tiempo.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF
  Desventajas — necesita conocer las ráfagas **de antemano** (un dato del futuro) y puede producir **inanición** del largo. 🖊 (→ U2: El proceso más corto a continuación (SPN)).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3 — Round Robin
  🖊 Es el heredero del **tiempo compartido** de la tercera generación (→ U1: Tema 7 — Historia por generaciones).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3b — Planificación por prioridades y múltiples colas
  🖊 **Inanición + candado = inversión de prioridades** (→ U2: Inversión de prioridades). Si el de prioridad 9 tiene el candado X y el de prioridad 1 lo pide, el más importante queda frenado por el menos importante. **En el parcial, nombrá las dos**: "inversión de prioridades causada por la inanición del proceso de prioridad baja".
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativ
  🖊 El umbral dinámico de espera activa de Karlin (1991) es esta misma idea aplicada a candados (→ U2: 10.3 — Sincronización de multiprocesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativ
  Dos y tres niveles — figuran en el programa pero **no están desarrollados en el Tanenbaum** de la cátedra (no hay que inventarlos). 🖊 En multiprocesadores sí aparece la *planificación de dos niveles* (→ U2: 10.4 — Planificación de multiprocesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real
  Tiempo real duro (plazos absolutos) contra **suave** (perder un plazo ocasional es tolerable) 🖊 (→ U1:

Tiempo real duro contra suave).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.5 — Política contra mecanismo

  🔗 **Es la misma regla del microkernel** (→ U1: Mecanismo en el kernel, política en el espacio de usuario) y vuelve en el servidor central de Tucker y Gupta en multiprocesadores (→ U2: 10.4 — Planificación de multiprocesadores).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.6 — Planificación de hilos

  Planificador — usuario: puede ser específico de la aplicación (en el servidor web, preferir al despachador); kernel: no conoce la aplicación, pero puede preferir un hilo **del mismo proceso** porque es más barato (afinidad). 🔗 (→ U2: 10.4 — Planificación de multiprocesadores).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.1 — Los filósofos comelones

  🔗 Es la técnica de prevención más usada: **ordenar los recursos** para que no pueda haber **espera circular**, una de las cuatro condiciones del interbloqueo de la **U6**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.1 — Los filósofos comelones

  🔔 **El `down(&s[i])` va FUERA del mutex**: si el filósofo se durmiera con el mutex tomado, nadie podría entrar a `probar()` para despertarlo. 🔗 (→ U2: La regla precisa sobre dormirse con un candado tomado).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.1 — Los filósofos comelones

  🔔 **El `up(&s[i])` de `probar(i)` se guarda** si el filósofo todavía no hizo su `down`: por eso no se pierde el aviso (→ U2: Un up sin destinatario SE GUARDA; un wakeup sin destinatario SE PIERDE.).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.1 — Los filósofos comelones

  🔗 **Mutex de mesa contra `estado[]`** es el mismo movimiento que hace el SMP: partir un candado grande en candados chicos (→ U2: 10.2 — Los tres tipos de SO multiprocesador).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  En los filósofos sí sería interbloqueo: F0 y F2 necesitan el mutex para llamar a `probar(1)` (→ U2: La solución buena (fig. 2-46)).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  En productor-consumidor con los `down` invertidos también: el consumidor necesita el mutex que tiene el productor dormido (→ U2: Productor-consumidor con tres semáforos).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  🔗 Es la puerta a los **interbloqueos de la U6** (espera circular por recursos).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.1 — Hardware de multiprocesador

  🔗 Es la **condición de carrera elevada a hardware**: en un uniprocesador la causaba el reloj; acá, otra CPU que escribe al mismo tiempo (→ U2: Tema 5 — Condiciones de carrera y regiones críticas).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.1 — Hardware de multiprocesador

  Multiprocesadores basados en directorios — una base de datos en hardware que dice dónde está cada línea de caché y en qué estado. *"Hasta un multiprocesador con memoria compartida realiza muchas operaciones de paso de mensajes de manera interna."* 🔗 (→ U2: Tema 7.4b — Paso de mensajes y barreras).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.1 — Hardware de multiprocesador

  🔗 La línea de caché es la unidad que produce el *thrashing* de TSL (→ U2: 10.3 — Sincronización de multiprocesadores); y es la misma caché de la U1 (→ U1: Jerarquía de memoria).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres tipos de SO multiprocesador

  🔗 **Tiene la misma forma que $\Sigma\, C_i/P_i \leq 1$** de tiempo real (→ U2: Condición de planificabilidad).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres

tipos de SO multiprocesador

🌤 **Candado grande = correcto y lento; candados chicos e independientes = correcto y rápido.** 🖊 Es el `estado[]` de Dijkstra (→ U2: La solución buena (fig. 2-46)).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  🖊 **Por eso se inventó Peterson (1981)**: para sincronizar **solo con software**, sin depender de un bus bloqueable (→ U2: ✓ Solución 4 — Peterson).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  🖊 La lista FIFO del MCS es la **cola de bloqueados del semáforo**, pero girando (→ U2: Tema 7.2 — Semáforos).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  Algoritmo de retrospección — saber de antemano cuánto va a durar el candado: es el punto de referencia **irrealizable** (el mismo papel que SJF y el reemplazo óptimo). 🖊 (→ U2: SJF (el trabajo más corto primero)).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  Umbral dinámico (Karlin y colaboradores, 1991) — se decide con la **historia observada**: si las 3 esperas previas fueron de ~200 µs, girar hasta 2 ms; si las 3 llegaron a los 2 ms, **conmutar de inmediato**. 🖊 **Es el promedio exponencial del SPN aplicado a candados** (→ U2: El proceso más corto a continuación (SPN)).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  ¿Procesos o hilos? — con hilos de usuario se planifica por proceso; con hilos de kernel, el kernel elige el hilo y el proceso *"sólo tiene un pequeño papel (o tal vez ninguno)"* 🖊 (→ U2: 8.6 — Planificación de hilos).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  🌤 **El `make` paralelo**: *"si se necesitan 10 compilaciones, no tiene caso programar 9 de ellas de inmediato y dejar la última para mucho después"*. Cuando el resultado depende del **último** hilo, hay que optimizar el **máximo**, no el promedio. 🖊 Misma lógica que la **barrera** (→ U2: Barreras).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  🖊 La cola global es la región crítica de la **lista de listos**, la que obliga a girar (→ U2: Espera activa contra conmutación).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  🖊 **Es la inversión de prioridades con varias CPU** (→ U2: Inversión de prioridades): la bandera es justo la información que en el tema 8 el planificador "no tenía". Primo barato de la herencia de prioridades.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  🖊 El 3º es otra vez partir un candado grande en candados chicos (→ U2: 10.2 — Los tres tipos de SO multiprocesador).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  Servidor central (Tucker y Gupta, 1989) — cada aplicación sondea cuántas CPUs puede usar y ajusta su número de hilos (el servidor web que baja de 10 a 5). 🖊 Política contra mecanismo (→ U2: 8.5 — Política contra mecanismo).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma

  Proceso ≠ hilo: el proceso agrupa, el hilo ejecuta; `fork` duplica, `execve` reemplaza; n fork = 2^n (→ U2: Tema 1 — Qué es un proceso).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma

  Tres estados y cuatro transiciones; fin de quantum → LISTO; no existe BLOQUEADO → EJECUCIÓN

(→ U2: Las 4 transiciones y quién provoca cada una).

- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Las 4 condiciones y cuál viola cada intento: interrupciones → 2, candado → 1, alternancia → 3, Peterson y TSL funcionan (→ U2: Las 4 condiciones de una buena solución).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Espera ocupada contra bloqueo; la inversión de prioridades es culpa de la espera ocupada (→ U2: Inversión de prioridades).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Semáforos: un `up` sin destinatario se guarda; `wakeup` y `signal` se pierden; productor-consumidor con tres semáforos y el orden de los `down` (→ U2: Productor-consumidor con tres semáforos).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  ★ **Planificación**: retorno, espera y respuesta; los algoritmos; $\Sigma C_i/P_i \leq 1$ (→ U2: Las métricas).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Problemas clásicos: Dijkstra deja comer a 2 como máximo; escritor con inanición si $I \leq D$; la regla de dormirse con el candado (→ U2: La regla precisa sobre dormirse con un candado tomado).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Multiprocesadores: candado grande contra candados chicos; dos dimensiones; pandillas (→ U2: 10.4 — Planificación de multiprocesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Round Robin con quantum 1 y 2: Gantt, métricas y **la cola de preparados en un instante t**, con el desempate de la cátedra (→ U2: 8.3 — Round Robin).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  SJN y SRTF/SRTN (→ U2: 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF).
- U2 — Procesos, hilos, concurrencia y planificación › Cierre de la U2 y lo que más se toma
  Comandos: `ps -ef`, `pgrep`, `kill -STOP` / `kill -CONT`, `strace` (→ U1: strace).

Anexo C — Todas las trampas de examen (📌)

- U1 — Introducción, hardware y estructura de los SO › Tema 1 — ¿Qué es un sistema operativo?
  📌 El modelo ingenuo ("si calcula el 20 %, con 5 procesos la CPU está al 100 %") es **demasiado optimista** porque supone que los 5 nunca esperan E/S al mismo tiempo.
- U1 — Introducción, hardware y estructura de los SO › Tema 2 — Repaso del hardware
  📌 **Instrucción privilegiada ejecutada en modo usuario**: el **hardware** mira el bit de modo del PSW y la **rechaza antes de ejecutarla**, generando un trap. Recién ahí el SO toma el control y, normalmente, mata al proceso. **La protección la impone el hardware; el SO la configura.**
- U1 — Introducción, hardware y estructura de los SO › Tema 3 — Las tres formas de hacer E/S
  📌 **"¿Cuál es más rápido?"** — ninguno. Es la trampa del tema.
- U1 — Introducción, hardware y estructura de los SO › Tema 4 — El arranque y el sector de booteo
  📌 **¿Por qué hacen falta dos cargadores?** El código del MBR tiene 446 bytes; un cargador secundario como **GRUB** ocupa **cientos de kilobytes** (unas mil veces más). En 446 bytes solo entra "buscar la partición activa y leer su primer sector".
- U1 — Introducción, hardware y estructura de los SO › Tema 4 — El arranque y el sector de booteo
  📌 **El MBR no lee ext4**: solo salta al primer sector de la partición activa. El que entiende el sistema de archivos es el **cargador secundario**.
- U1 — Introducción, hardware y estructura de los SO › Tema 5 — Ensamblador e interrupciones del 8086
  📌 Trampa de la cátedra: *"No todo va en AH"*. Los datos pueden ir en AL, DL, DX, BX, CX, DS:DX, etc.
- U1 — Introducción, hardware y estructura de los SO › Tema 5 — Ensamblador e interrupciones del 8086
  📌 La cartilla marca que 09h, 0Ah, 0Eh y 13h son las más fáciles de confundir.
- U1 — Introducción, hardware y estructura de los SO › Tema 5 — Ensamblador e interrupciones del 8086
  📌 **Los 4 errores típicos que marca la cátedra**: usar **DL** en lugar de DX (DL es para un carácter, función 02h); olvidar el **$** (DOS muestra basura hasta encontrar un $ por casualidad); **no cargar DS**; usar **AH = 02h** para una cadena (muestra un solo carácter).
- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema

📌 **La llamada puede bloquear al proceso**: si se lee del teclado y no hay nada, el proceso se bloquea, el SO corre otro, y los pasos 9 a 11 se completan cuando llega la entrada. 🔗 Transición ① del diagrama de estados (→ U2: Tema 2 — Estados del proceso y el BCP).

- U1 — Introducción, hardware y estructura de los SO › Tema 6 — Llamadas al sistema
  📌 Sintaxis exacta que piden: **`strace -o salida.txt ls -la`**. Distractores habituales: `>`, `|`, `-s`.
- U1 — Introducción, hardware y estructura de los SO › Tema 7 — Historia por generaciones
  📌 **La multiprogramación exige hardware de protección** (si no, un trabajo pisa la memoria de otro). Recién lo trajo la tercera generación (IBM 360). 🔗 (→ U1: Modo kernel y modo usuario).
- U1 — Introducción, hardware y estructura de los SO › Tema 8 — Tipos de sistemas operativos
  Tiempo real duro contra suave — 📌 se toma.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso
  📌 Tres ventanas del mismo editor = **1 programa y 3 procesos**. Comparten el **código**; cada uno tiene su PID, su espacio de direcciones, sus variables, su PC, su pila, sus registros y sus archivos abiertos.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso
  📌 50 procesos en un solo núcleo es un sistema **concurrente**, no **paralelo**. 🔗 El paralelismo real es el tema 10 (→ U2: Tema 10 — Sistemas de múltiples procesadores).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 1 — Qué es un proceso
  📌 Con un `if (fork() == 0) { fork(); }` ya no es 2^n: hay que seguir el árbol rama por rama.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  📌 **No existe Bloqueado → Ejecución.** Cuando llega el dato, el proceso pasa a LISTO y espera su turno: **④ y después ③, nunca una sola transición.**
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  📌 **Tampoco existe Listo → Bloqueado**: para bloquearse hay que *pedir* algo, y para pedir hay que estar ejecutándose.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  📌 Un proceso de más prioridad que se desbloquea **no salta** directo a ejecución: el planificador aplica ② al actual y ③ al desbloqueado (dos transiciones), y en un sistema **no apropiativo** ni siquiera eso.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 2 — Estados del proceso y el BCP
  📌 Guardar los registros y cambiar la pila **no se puede escribir en C**: por eso esa parte es ensamblador.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico
  📌 **Por qué cada hilo necesita su propia pila**: cada uno tiene un historial de llamadas distinto. Con una pila compartida, el `return` de un hilo desapilaría el marco del otro. 🔗 Es la pila del CALL/RET de la U1 (→ U1: Tema 6 — Llamadas al sistema).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 3 — Hilos: el modelo clásico
  📌 `close(fd)` en un hilo: el descriptor es del proceso, así que **desaparece para todos**; si otro hilo estaba leyendo, su `read()` falla. Con procesos separados, cada uno tiene su tabla de descriptores.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 4 — Implementación de hilos
  📌 **La paradoja**: los programadores quieren hilos justamente en las aplicaciones que hacen muchas llamadas bloqueantes, que es donde los hilos de usuario funcionan peor.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  📌 **Lo que libera al que gira no es un cambio de `turno`**: es que el otro ponga `interesado[otro] = FALSE` al salir.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  📌 **TSL bloquea el BUS DE MEMORIA**, no "a las otras CPU": ninguna CPU puede acceder a la memoria hasta que termine. Deshabilitar interrupciones bloquea **una sola CPU**.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 6 — Exclusión mutua con espera ocupada
  📌 **"Girar 1000 veces y después dormir un rato"** no lo resuelve del todo: no hay cómo elegir cuánto dormir sin suponer velocidades; se pierde la garantía de prioridad; se gastan igual 1000 vueltas; y con un tercer proceso M de prioridad media se vuelve a colgar (**dormir garantiza que H suelte la CPU, no que L la reciba**). 🔔 **Dormir por tiempo ≠ dormir por evento**; lo segundo es el semáforo (→ U2: Tema 7.2 — Semáforos).
- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos
  📌 **Eso no es espera ocupada**: la operación del semáforo dura microsegundos; la región crítica dura lo

que dure.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.2 — Semáforos

  📌 **El orden de los `down` NO es decorativo**: si el productor hace `down(&mutex)` antes que `down(&vacias)` con el búfer lleno, **se duerme con el mutex puesto** y el consumidor nunca puede entrar a sacar: **interbloqueo**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.3 — Mutexes

  📌 **Las variables de condición NO tienen memoria**: una señal sin nadie esperando **se pierde** (como el `wakeup`). Por eso van siempre con un mutex.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4a — Monitores

  📌 **Es una construcción DEL LENGUAJE**: lo garantiza **el compilador**. *"Sólo puede haber un proceso activo en un monitor en cualquier instante."* (C no tiene monitores; el libro usa Pidgin Pascal.)

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4a — Monitores

  📌 **El libro adopta Brinch Hansen**, *"debido a que, en concepto, es más simple y también más fácil de implementar"*: con esa regla, **`signal` tiene que ser la última instrucción** del procedimiento de monitor.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4a — Monitores

  📌 **Las variables de condición no son contadores**: la señal sin oyente **se pierde**, así que `wait` tiene que ir **antes** que `signal`.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 7.4b — Paso de mensajes y barreras

  📌 **No requieren memoria compartida**: es el único mecanismo que sirve entre máquinas distintas.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas

  📌 **El factor clave es la longitud de la ráfaga de CPU, NO la de la E/S.** *"Están limitados a la E/S debido a que no realizan muchos cálculos entre una petición de E/S y otra, no debido a que tengan peticiones de E/S en especial largas."*

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.1 — Conceptos y métricas

  📌 `inicio - llegada` como espera **solo sirve sin apropiación**.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF

  📌 **SJF es óptimo SOLO cuando todos los trabajos están disponibles al mismo tiempo.** Contraejemplo del libro: A–E con 2, 4, 1, 1, 1 que llegan en 0, 0, 3, 3, 3. SJF hace A, B, C, D, E → promedio 4,6; hacer B, C, D, E, A da 4,4.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.3c — Proceso más corto a continuación, garantizada, lotería y partes equitativ

  📌 **¿Por qué no se puede hacer con prioridades?** Porque una prioridad es un **puesto (ordinal)**, no una **cantidad** divisible y transferible ("sos 3º", no "tenés 15"); y **la prioridad la fija el sistema**, no el proceso. Los boletos son tuyos y los regalás.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real

  📌 **Trampa 1 — cuál va arriba**: es C_i / P_i (CPU que necesita / cada cuánto ocurre). Cada término tiene que dar menos que 1; si uno da más, lo invertiste (o ese evento solo ya es imposible).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real

  📌 **Trampa 2 — unidades**: si los períodos están en segundos y la CPU en milisegundos, la cuenta sale 1000 veces mal. Pasar todo a la misma unidad antes de dividir.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real

  📌 **Los aperiódicos no están en la suma**: si llega una emergencia, come del margen libre.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real

  📌 **Los C_i tienen que ser del PEOR caso**, no del promedio.

- U2 — Procesos, hilos, concurrencia y planificación › Tema 8 — Planificación de la CPU ★ › 8.4 — Planificación de tiempo real

  📌 **La condición es NECESARIA pero NO SUFICIENTE.** Ejemplo: A (cada 100 ms, 50 ms) y B (cada 10 ms, 2 ms) suman 0,70, pero con un planificador **no apropiativo** que arranca A, B termina en t = 52 y pierde su plazo. Hace falta capacidad **y** un algoritmo que la use bien (apropiativo, con prioridad al plazo

más cercano).

- U2 — Procesos, hilos, concurrencia y planificación › Tema 9 — Problemas clásicos de comunicación entre procesos › 9.2 — Lectores y escritores

  📌 **La "simplificación" de que cada lector haga `down(&bd)` sin `cl`** es correcta (nadie corrompe nada) pero **serializa a los lectores**: entran de a uno, y se pierde la razón de ser del problema. Es el mismo error del mutex de toda la mesa: un candado demasiado grande.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.1 — Hardware de multiprocesador

  📌 *"Aun con la mejor caché, el uso de un solo bus limita el tamaño de un multiprocesador UMA a 16 o 32 CPUs."* El límite es del **bus único**: con barras cruzadas UMA escala más.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres tipos de SO multiprocesador

  📌 **Cuello de botella** — *"si el 10 % del tiempo está manejando llamadas al sistema, con 10 CPUs el maestro casi se saturará y con 20 estará completamente sobrecargado"*.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.2 — Los tres tipos de SO multiprocesador

  📌 **"SMP resuelve el cuello de botella del maestro-esclavo" es FALSO si el SO tiene un solo candado global.** Lo que lo resuelve es la **granularidad de los candados**.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  📌 **Si dos ven libre a la vez no es una condición de carrera**: *"la verdadera adquisición la realiza la instrucción TSL, y esta instrucción es atómica"*; *"el éxito en la lectura pura es tan sólo una sugerencia"*.
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.3 — Sincronización de multiprocesadores

  📌 **A veces no hay opción**: el candado de la **lista de procesos listos** obliga a girar, *"ya que para ello tendría que leer la lista de procesos listos"*. **No se puede conmutar para poder conmutar.**
- U2 — Procesos, hilos, concurrencia y planificación › Tema 10 — Sistemas de múltiples procesadores › 10.4 — Planificación de multiprocesadores

  📌 **Ahora hay DOS dimensiones**: *qué hilo ejecutar* **y** *en cuál CPU*. En un uniprocesador había una sola. Es la respuesta de una línea a "¿qué cambia en la planificación?".